



Newton Application Development

Newton Internet Enabler



Version 1.1

© Apple Computer, Inc. 1996

 Apple Computer, Inc.
© 1996, Apple Computer, Inc.
All rights reserved.

No part of this publication or the software described in it may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, mechanical, electronic, photocopying, recording, or otherwise, without prior written permission of Apple Computer, Inc., except in the normal use of the software or to make a backup copy of the software. The same proprietary and copyright notices must be affixed to any permitted copies as were affixed to the original. This exception does not allow copies to be made for others, whether or not sold, but all of the material purchased (with all backup copies) may be sold, given, or loaned to another person. Under the law, copying includes translating into another language or format. You may use the software on any computer owned by you, but extra copies cannot be made for this purpose.

Printed in the United States of America.

The Apple logo is a registered trademark of Apple Computer, Inc. Use of the "keyboard" Apple logo (Option-Shift-K) for commercial purposes without the prior written consent of Apple may constitute trademark infringement and unfair competition in violation of federal and state laws.

No licenses, express or implied, are granted with respect to any of the technology described in this book. Apple retains all intellectual property rights associated with the technology described in this book. This book is intended to assist application developers to develop

applications only for Newton devices.

Apple Computer, Inc.
1 Infinite Loop
Cupertino, CA 95014
408-996-1010

Apple, the Apple logo, APDA, AppleLink, LaserWriter, Macintosh, Mac, MPW, MessagePad, and Newton are trademarks of Apple Computer, Inc., registered in the United States and other countries.

Adobe Illustrator and PostScript are trademarks of Adobe Systems Incorporated, which may be registered in certain jurisdictions. FrameMaker is a registered trademark of Frame Technology Corporation.

Helvetica and Palatino are registered trademarks of Linotype Company.

ITC Zapf Dingbats is a registered trademark of International Typeface Corporation.

Varityper is a registered trademark of Varityper, Inc.

Windows is a trademark of Microsoft, Inc.

Simultaneously published in the United States and Canada.

LIMITED WARRANTY ON MEDIA AND REPLACEMENT

If you discover physical defects in the manual or in the media on which a software product is distributed, APDA will replace the media or manual at no charge to you provided you return the item to be replaced with proof of purchase to APDA.

ALL IMPLIED WARRANTIES ON THIS MANUAL, INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, ARE LIMITED IN DURATION TO NINETY

(90) DAYS FROM THE DATE OF THE ORIGINAL RETAIL PURCHASE OF THIS PRODUCT.

Even though Apple has reviewed this manual, APPLE MAKES NO WARRANTY OR REPRESENTATION, EITHER EXPRESS OR IMPLIED, WITH RESPECT TO THIS MANUAL, ITS QUALITY, ACCURACY, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE. AS A RESULT, THIS MANUAL IS SOLD "AS IS," AND YOU, THE PURCHASER, ARE ASSUMING THE ENTIRE RISK AS TO ITS QUALITY AND ACCURACY.

IN NO EVENT WILL APPLE BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES RESULTING FROM ANY DEFECT OR INACCURACY IN THIS MANUAL, even if advised of the possibility of such damages.

THE WARRANTY AND REMEDIES SET FORTH ABOVE ARE EXCLUSIVE AND IN LIEU OF ALL OTHERS, ORAL OR WRITTEN, EXPRESS OR IMPLIED. No Apple dealer, agent, or employee is authorized to make any modification, extension, or addition to this warranty.

Some states do not allow the exclusion or limitation of implied warranties or liability for incidental or consequential damages, so the above limitation or exclusion may not apply to you. This warranty gives you specific legal rights, and you may also have other rights which vary from state to state.

Related Books	vii
Sample Code	viii
Conventions Used in This Book	ix
Special Fonts	ix
Tap Versus Click	ix
Frame Code	x
Developer Products and Support	xi
Undocumented System Software Objects	xii

About Newton Internet Enabler	1-2
The Inet Tool Layer	1-4
The Link Controller	1-4
The Domain Name Service Interface	1-6
Using Newton Internet Enabler	1-7
Using Endpoints With Newton Internet Enabler Links	1-8
Using Multiple Endpoints With a Link	1-9
Newton Internet Enabler and Callback Functions	1-9
Using the Newton Internet Enabler Link Controller	1-11
Grabbing a Link	1-12
Retrieving and Displaying Link Status Information	1-15
Configuring Newton Internet Enabler for Your Endpoint	1-16
Binding Your Endpoint with Newton Internet Enabler	1-17
Connecting Your Endpoint with Newton Internet Enabler	1-18
Sending Data	1-19
Sending Data With a UDP Connection	1-19
Sending Data With a TCP Connection	1-20
Receiving Data	1-21
Receiving Data With UDP	1-22
Receiving Data With TCP	1-24
Disconnecting Your Endpoint	1-25
Releasing Your Link	1-26
Power Management and Internet Links	1-27
Using the Domain Name Service Interface	1-28
Using the Newton Internet Enabler Options	1-29
Newton Internet Enabler Interface Reference	1-31
Constants	1-31
Status Code Constants	1-31
Transport Service Type Constants	1-31
Link Controller Error Codes	1-32
DNS Error Codes	1-32
Newton Internet Enabler PPP Errors	1-35
Newton Internet Enabler Lower-Level Tool Errors	1-35

Inet Tool Errors	1-35
Application-related Errors	1-38
UDP Errors	1-41
Inet Tool-Specific Errors	1-42
PPP Internal Errors	1-43
Newton Internet Enabler Events	1-46
Handling TCP Disconnect Events	1-48
Newton Internet Enabler Function Parameter Information	1-48
The Link Identifier Parameter	1-48
The Client Context Parameter	1-49
The Client Callback Parameter	1-49
The Link Controller Callback Function Format	1-50
The Link Controller Status Frame	1-50
The Link Entry Information Frame	1-51
Login Script Frames	1-54
The Domain Name Service Callback Function Format	1-55
The DNS Results Frame	1-56
Link Controller Functions and Methods	1-58
InetAddNewLinkEntry	1-58
InetAddNewLinkEntryInternal	1-58
InetCancelCurrentRequest	1-59
InetCancelLink	1-59
InetDisconnectLink	1-60
InetDisplayStatus	1-60
InetGetAllLinksStatus	1-62
InetGetDefaultLinkID	1-63
InetGetLinkEntry	1-63
InetGetLinkStatus	1-63
InetGrabLink	1-64
InetOpenConnectionSlip	1-64
InetReleaseLink	1-65
InetSetDefaultLinkID	1-66
Domain Name Service Functions and Methods	1-67
DNSCancelRequests	1-67
DNSGetAddressFromName	1-67
DNSGetMailAddressFromName	1-69
DNSGetMailServerNameFromDomainName	1-70
DNSGetNameFromAddress	1-71
Newton Internet Enabler Error Functions	1-72
InetGetErrorString	1-72
InetGetExceptionError	1-73
Newton Internet Enabler Options	1-73
Inet Tool Expedited Data Transfer ('iexp') Option	1-73
Inet Tool Physical Link Identifier ('ilid') Option	1-74
Inet Tool Local Port ('ilpt') Option	1-75
Inet Profile ('iprf') Option	1-77
Inet Tool TCP Remote Socket ('itrs') Option	1-78

Inet Tool Transport Service Type ('itsv') Option	1-79
Inet Tool UDP Destination Socket ('iuds') Option	1-80
Inet Tool UDP Source Socket ('iuss') Option	1-81
Newton Internet Enabler Exceptions	1-83
Summary of Newton Internet Enabler	1-83
Link Controller Errors	1-83
DNS Errors	1-83
Newton Internet Enabler PPP Errors	1-84
Newton Internet Enabler Lower-Level Tool Errors	1-84
Link Controller Functions and Methods	1-87
DNS Functions and Methods	1-87
Error-handling Functions and Methods	1-88
Exceptions	1-88

About This Document

This document describes how to use the Newton Internet Enabler, a collection of built-in software that allows Newton devices to interface with the Internet.

Related Books

This book is one in a set of books available for Newton programmers. You'll also need to refer to these other books in the set:

- *Newton Programmer's Guide*. This book is the definitive guide to Newton programming, providing conceptual information and instructions for using the Newton application programming interfaces.
- *Newton Programmer's Reference*. This online book is the comprehensive reference to the Newton programming interface. It documents all routines, prototypes, data structures, constants, and error codes defined by the Newton system for use by NewtonScript developers.
- *Newton Toolkit User's Guide*. This book comes with the Newton Toolkit development environment. It introduces the Newton development environment and shows how to develop applications using Newton Toolkit. You should read this book first if you are a new Newton application developer.
- *The NewtonScript Programming Language*. This book comes with the Newton Toolkit development environment. It describes the NewtonScript programming language.
- *Newton Book Maker User's Guide*. This book comes with the Newton Toolkit development environment. It describes how to

use Newton Book Maker and Newton Toolkit to make Newton digital books and to add online help to Newton applications.

- *Newton 2.0 User Interface Guidelines.* This book contains guidelines to help you design Newton applications that optimize the interaction between people and Newton devices.

Sample Code

There is sample code that deals specifically with the Newton Internet Enabler.

The Newton Toolkit development environment, from Apple Computer, includes many sample code projects. You can examine these samples, learn from them, and experiment with them. These sample code projects illustrate most of the topics covered in this book. They are an invaluable resource for understanding the topics discussed in this book and for making your journey into the world of Newton programming an easier one.

The Newton Developer Technical Support team continually revises the existing samples and creates new sample code. The latest sample code is included each quarter on the Newton Developer CD, which is distributed to all Newton Developer Program members and to subscribers of the Newton monthly mailing. Sample code is updated on the Newton Development side on the World Wide Web (<http://devworld.apple.com/dev/newtondev.shtml>) shortly after it is released on the Newton Developer CD. For information about how to contact Apple Computer regarding the Newton Developer Program, see the section “Developer Products and Support,” on page xi.

The code samples in this book show methods of using various routines and illustrate techniques for accomplishing particular tasks. All code samples have been compiled and, in most cases,

tested. However, Apple Computer does not intend that you use these code samples in your application.

To make the code samples in this book more readable, only limited error handling is shown. You need to develop your own techniques for detecting and handling errors.

Conventions Used in This Book

This book uses the following conventions to present various kinds of information.

Special Fonts

This book uses the following special fonts:

- **Boldface.** Key terms and concepts appear in boldface on first use.
- **Courier typeface.** Code listings, code snippets, and special identifiers in the text such as predefined system frame names, slot names, function names, method names, symbols, and constants are shown in the Courier typeface to distinguish them from regular body text. If you are programming, items that appear in Courier should be typed exactly as shown.
- *Italic typeface.* Italic typeface is used in code to indicate replaceable items, such as the names of function parameters, which you must replace with your own names. The names of other books are also shown in italic type, and *rarely*, this style is used for emphasis.

Tap Versus Click

Throughout the Newton software system and in this book, the word “click” sometimes appears as part of the name of a method or variable, as in `ViewClickScript` or `ButtonClickScript`. This may lead you to believe that the text refers to mouse clicks. It

does not. Wherever you see the word “click” used this way, it refers to a tap of the pen on the Newton screen (which is somewhat similar to the click of a mouse on a desktop computer).

Frame Code

If you are using the Newton Toolkit (NTK) development environment in conjunction with this book, you may notice that this book displays the code for a frame (such as a view) differently than NTK does.

In NTK, you can see the code for only a single frame slot at a time. In this book, the code for a frame is presented all at once, so you can see all of the slots in the frame, like this:

```
{  viewClass: clView,
    viewBounds: RelBounds( 20, 50, 94, 142 ),
    viewFlags: vNoFlags,
    viewFormat: vfFillWhite+vfFrameBlack+vfPen(1),
    viewJustify: vjCenterH,

    ViewSetupDoneScript: func()
        :UpdateDisplay(),

    UpdateDisplay: func()
        SetValue(display, 'text, value);
};
```

If while working in NTK, you want to create a frame that you see in the book, follow these steps:

1. On the NTK template palette, find the view class or proto shown in the book. Draw out a view using that template. If the frame shown in the book contains a `_proto` slot, use the corresponding proto from the NTK template palette. If the frame shown in the book contains a `viewClass` slot instead of

a `_proto` slot, use the corresponding view class from the NTK template palette.

2. Edit the `viewBounds` slot to match the values shown in the book.
3. Add each of the other slots you see listed in the frame, setting their values to the values shown in the book. Slots that have values are attribute slots, and those that contain functions are method slots.

Developer Products and Support

The *Apple Developer Catalog* (ADC) is Apple Computer's worldwide source for hundreds of development tools, technical resources, training products, and information for anyone interested in developing applications on Apple computer platforms. Customers receive the *Apple Developer Catalog* featuring all current versions of Apple development tools and the most popular third-party development tools. ADC offers convenient payment and shipping options, including site licensing.

To order product or to request a complimentary copy of the *Apple Developer Catalog* contact

Apple Developer Catalog
 Apple Computer, Inc.
 P.O. Box 319
 Buffalo, NY 14207-0319

Telephone	1-800-282-2732 (United States) 1-800-637-0029 (Canada) 716-871-6555 (International)
-----------	---

Fax	716-871-6511
-----	--------------

AppleLink	ORDER.ADC
-----------	-----------

Internet	order.adc@applelink.apple.com
----------	-------------------------------

World Wide Web	http://www.devcatalog.apple.com
----------------	---

If you provide commercial products and services, call 408-974-4897 for information on the developer support programs available from Apple.

For Newton-specific information, see the Newton developer World Wide Web page at:

<http://devworld.apple.com/dev/newtondev.shtml>

Undocumented System Software Objects

When browsing in the NTK Inspector window, you may see functions, methods, and data objects that are not documented in this book. Undocumented functions, methods, and data objects are not supported, nor are they guaranteed to work in future Newton devices. Using them may produce undesirable effects on current and future Newton devices.

Newton Internet Enabler

This chapter describes the NewtonScript interface to Newton Internet Enabler, a collection of built-in software that allows Newtons to interface with the Internet. This chapter describes:

- how the components of Newton Internet Enabler work together
- how to use Newton Internet Enabler in your NewtonScript applications
- the constants, protos, functions, and methods that you use with Newton Internet Enabler
- the options that you use to control and configure Newton Internet Enabler

For information about the user interface to Newton Internet Enabler, see the *User's Guide to Newton Internet Enabler*.

To use Newton Internet Enabler, you must understand how to use endpoints to perform communications operations on the Newton. To learn about endpoints, see the chapter “Endpoint Interface” in *Newton Programmer's Guide*.

About Newton Internet Enabler

With Newton Internet Enabler, you can establish a link to the Internet to allow your applications to communicate over the net. Newton Internet Enabler supports one link to the Internet at any time; however, several Newton applications can use that link to perform communications.

Newton Internet Enabler consists of an application named Internet Setup, with which users can configure their Internet access, and a NewtonScript application programming interface (API), with which you can send and receive data, determine status, and modify your configuration parameters.

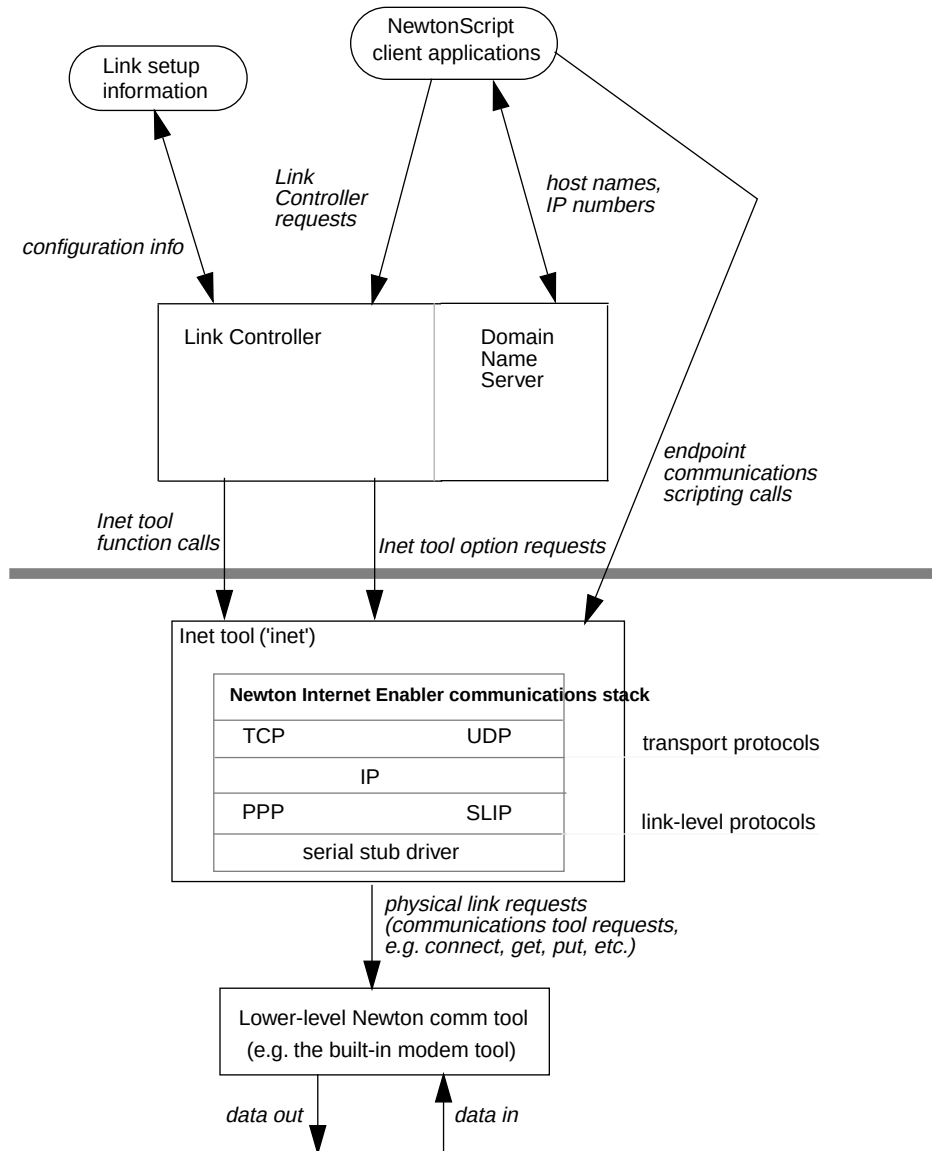
Underneath the NewtonScript API is the Inet tool, which is a built-in Newton communications tool that is used by the other Newton Internet Enabler components to provide the standard Newton communications operations.

The Newton Internet Enabler interface components are:

- The Inet communications tool, which provides the capability to establish and use links to the Internet via the TCP/IP family of protocols. This tool implements a streams interface with a communications stack, providing the capability to use various transport-level and link-level protocols. You can use standard Newton communications endpoint methods and options to control the operations provided by the Inet tool.
- The Link Controller, which uses the Inet tool and maintains net links at a higher level, providing a convenient NewtonScript interface for applications. You use the link controller by calling the global functions that it provides.
- The Domain Name Service interface, which provides functions for converting between Internet host names and their corresponding IP Numbers. You use the domain name service by calling the global functions that it provides.

Figure 1-1 shows the relationship of the Newton Internet Enabler components.

Newton Internet Enabler

Figure 1-1 The Newton Internet Enabler layers and components

The Inet Tool Layer

The Inet communications tool provides a configurable stack of protocols at and below the TCP/IP level. The Inet tool is a standard Newton communications tool, which means that it provides all of the endpoint services that are provided by other built-in communications tools, including the built-in modem tool and the built-in serial tool.

Like the other communications tools, you can control the configuration of the Inet tool with communications options. The options that you use with the Inet tool all use the 'inet' service identifier and are described in this chapter.

For more information on how to use the Newton endpoint interface, Newton communications options, and the other built-in communications tools, see *Newton Programmer's Guide*.

The Inet tool supports physical links using the built-in serial tool or the built-in modem tool. You are currently limited to the use of one physical link at any point in time.

The Inet tool can run various link-level protocols that are provided with the Newton system software. These currently include PPP and SLIP.

The Inet tool can establish links using various low-level communications services. Each communications service is provided by a Newton communications tool such as the built-in modem tool.

The Link Controller

You can use the Link Controller to create and manage a link between the Newton and the Internet. The Link Controller can manage a single link for multiple applications simultaneously. This means that one application establishes the link and other applications use the same link.

The Link Controller uses and provides a higher-level interface to the link control functions and options supported by the Inet tool. The Link Controller functions are global functions built into the Newton operating system. The

Newton Internet Enabler

use of the link controller functions is explained in the section “Using the Newton Internet Enabler Link Controller” beginning on page 1-11.

The first grab of a link can be expensive in terms of time: typically, the Inet tool software dials the Newton modem and negotiates the connection to establish an Internet session. The Inet tool then performs whatever login and initialization procedures are required, which the user has configured with the Internet Setup application. All of this can take a substantial amount of time.

Since it can take so much time to grab a new link, Newton Internet Enabler makes it easy for another application to grab a link that has already been established. Whenever an application grabs a link, the link controller increments its count of users of that link. The physical link is dropped only after all users have released the link (when the count becomes 0).

Note

Dropping of the physical link is also dependent on the hang-up time preferences set by the user with the Internet Setup application. ♦

The following is an example of a typical flow of operations that occur during an Internet session:

1. An application (“Application_1”) calls `InetOpenConnectionSlip` to allow the user to customize the link settings and then issues a call to the `InetGrabLink` function. The link controller dials the modem and begins an Internet session with an Internet provider.
2. Application_1 instantiates and binds one or more endpoints to use over that link. Each endpoint can use either the TCP or UDP transport services. And you bind each endpoint either to initiate an outgoing connection (*connect*) or to listen for an incoming connection (*listen*).
3. Application_1 uses its endpoint(s) to perform communications operations.
4. Another application (“Application_2”) calls `InetOpenConnectionSlip` to allow the user to customize the link settings and then issues a call to the `InetGrabLink` function to use the same service provider as Application_1. The Inet tool returns the same link that it established in step 1.

Newton Internet Enabler

5. Application_2 creates and uses endpoint(s) to perform communications operations.
6. Application_2 finishes its use of the link and calls the `InetReleaseLink` function. The link controller decrements its count of users of the link.
7. Application_3 grabs the link, creates endpoints to use over the link, and releases the link.
8. Application_1 finishes its use of the link and calls the `InetReleaseLink` function. The link controller decrements its count of link users. The count becomes 0, so the link is dropped: the Internet session ends, the modem is hung up, and any resources used for the link are released.

Note

Only one Internet session can be active at any time. This means that if an application requests a link to a different Internet service provider when a session is in progress, the `InetGrabLink` call will fail and generate an error. ♦

The Domain Name Service Interface

The domain name service interface builds on top of the `Inet` tool to provide Newton applications with the ability to translate Internet domain names into IP numbers and vice-versa. The domain name service (DNS) functions are also global functions in the Newton operating system. You can use these functions to:

- translate a domain name into its corresponding Internet address
- translate a a domain name into the Internet address for a mail server that serves that domain
- translate a domain name into the domain name for a mail server that serves that domain
- translate an Internet address into its corresponding domain name

Newton Internet Enabler

IMPORTANT

The Newton Internet Enabler implements what is known as a “stub domain name service resolver.” The NIE domain name resolver does not re-query the server based on what type of response it received. This means that users must specify a recursive name server for their DNS server.

The use of the domain name server functions is explained in the section “Using the Domain Name Service Interface” beginning on page 1-28.

Using Newton Internet Enabler

This section helps you understand how to use Newton Internet Enabler in your NewtonScript application programs. To use Newton Internet Enabler, you need to know about the options and functions provided by the Newton Internet Enabler API. You also need to understand the callback functions that many of the functions use to communicate results back to your application. And finally, you need to know the proper sequence of steps to take when using Newton Internet Enabler.

The next section, “Using Endpoints With Newton Internet Enabler Links,” describes the relationship between the links that Newton Internet Enabler maintains for you and Newton communications endpoints.

The section, “Newton Internet Enabler and Callback Functions” beginning on page 1-9 describes the format and use of callback functions with Newton Internet Enabler.

The section “Using the Newton Internet Enabler Link Controller” beginning on page 1-11 outlines the basic sequence of steps that you need to perform to use the link controller with a Newton communications endpoint. The subsections of “Using the Newton Internet Enabler Link Controller” describe each step in detail.

The section “Using the Domain Name Service Interface” beginning on page 1-28 describes how to use the domain name service API to translate between Internet addresses and domain name strings.

Newton Internet Enabler

The section “Using the Newton Internet Enabler Options” beginning on page 1-29 provides detailed information about using Newton communications options to configure and control your Newton Internet Enabler sessions.

Note

You can override the default Inet icon by defining the 'Icon slot in your application's base view. ♦

Using Endpoints With Newton Internet Enabler Links

You use Newton Internet Enabler in your applications in much the same way that you use the other built-in Newton communications tools: you instantiate endpoints to use with Newton Internet Enabler and perform your communications operations with those endpoints. To learn about endpoints, see the chapter “Endpoint Interface” in *Newton Programmer's Guide*.

You specify the Newton Internet Enabler service identifier ('inet') in your service options, and configure Newton Internet Enabler by passing options in options frames to your endpoint methods. See the section “Using the Newton Internet Enabler Options” beginning on page 1-29 for information about which options to use with each of your endpoint methods.

You can use a Newton Internet Enabler link with more than one endpoint. In fact, it makes sense to reuse your link with multiple endpoints over the life of your application: the first application to grab the link establishes an Internet session by dialing a modem and negotiating the low-level connection, and subsequent applications can use the same session without having to pay that price.

Your application can use several endpoints with the same Newton Internet Enabler link. Each endpoint, however, requires a significant amount of memory. And the Newton system software restricts the total number (for all applications) of endpoints that can be active.

To use multiple endpoints in your application, follow this sequence of operations:

Newton Internet Enabler

1. Grab the link, as described in the section “Grabbing a Link” beginning on page 1-12.
2. Instantiate your first endpoint for use with Newton Internet Enabler. Use this endpoint to perform communications and then dispose of the endpoint.
3. Instantiate, use, and dispose of other endpoints.
4. Release your Newton Internet Enabler link, as described in the section “Releasing Your Link” beginning on page 1-26.

Using Multiple Endpoints With a Link

Several Newton applications can use endpoints at the same time. In fact, multiple applications can use multiple endpoints with a single Newton Internet Enabler link. However, there are certain restrictions around endpoint usage that you must beware of:

- Each endpoint requires a significant amount of memory
- The number of endpoints that can be open on the Newton at any time is limited. The exact limit depends on your hardware configuration.
- The limit on opened endpoints applies to the Newton as a whole. This means that if the machine limit is 4 endpoints and one application is using 2 endpoints, all other applications will be restricted to the possibility of using 2 endpoints.

Newton Internet Enabler and Callback Functions

Many of the Newton Internet Enabler functions require you to provide a **callback function**, which is a function that the Inet tool calls during and/or after the performance of the operation that you requested. The callback function receives status and error information.

For example, the `InetCancelLink` function calls the callback function that you provide after it finishes its operation. Your callback function for `InetCancelLink` can determine if an error occurred and can determine the current status of the link that you wanted cancelled.

Newton Internet Enabler

Some operations call your callback function more than once. For example, the `InetGrabLink` function calls the callback function you provide many times during its operations. You can use your `InetGrabLink` callback function to monitor the progress of the grab, since each call to it provides you with the current status.

When a function requires that you specify a callback function, you do so by providing a context frame and the symbol of the function defined in that frame that you want to use as the callback function. For example, the `InetGrabLink` function takes three parameters and is declared as follows:

```
InetGrabLink(linkID, clientContext, clientCallback);
```

When you call `InetGrabLink`, you must specify a frame (or your application frame) as the value of *clientContext*, and you must specify a function defined in the frame as the value of *clientCallback*.

You might create a callback function for your `InetGrabLink` calls that looks like the following:

```
myApp.GrabLinkCallback := func(linkID, stat, err)
begin
  if err=nil and stat.linkStatus <> 'connected then
    ;      // display status
  if err then
    ;      //handle the error
  // link established, so resolve the address
end;
```

Then, when you call the `InetGrabLink` function in your application, you pass it the name of your callback function. For example:

```
myApp.TestGrab := func()
begin
  myStatusView := InetStatusDisplay(nil, nil, nil);
  InetGrabLink(nil, self, 'GrabLinkCallback);
  ...
end;
```

Newton Internet Enabler

This function first calls the `InetStatusDisplay` function to create and display the status view. The call to `InetGrabLink` uses the default link ID and specifies `self` (the application frame) as the value of the *clientContext* parameter, and `'GrabLinkCallback'` (the symbol for the callback function) as the value of the *clientCallback* parameter. The `GrabLinkCallback` function will be called repeatedly while the system is attempting to grab the link, until either the status is `'connected'` or an error occurs.

The section “Grabbing a Link” beginning on page 1-12 provides a complete example and explanation of a callback function for the `InetGrabLink` function.

The section “Retrieving and Displaying Link Status Information” beginning on page 1-15 describes how to use the `InetStatusDisplay` function to display the progress of your `InetGrabLink` operation.

The section “Newton Internet Enabler Function Parameter Information” beginning on page 1-48 provides a detailed description of the *clientContext* and *clientCallback* parameters that you use in your Newton Internet Enabler function calls.

Using the Newton Internet Enabler Link Controller

The following is the sequence of steps that you need to perform when using Newton Internet Enabler in your application. Each step is described in detail in the sections that follow.

1. Grab a link by calling the `InetGrabLink` function. Before calling `InetGrabLink`, call `InetOpenConnectionSlip` to allow the user to modify the default link settings. While the link is being grabbed, call the `InetDisplayStatus` function to report the current link status to the user. After the link is grabbed, store the returned link ID into a variable for future access.
2. Instantiate an endpoint, passing down the Inet configuration options.
3. Bind the endpoint. The options that you pass down depend on whether you are using TCP or UDP, and on whether you are binding to initiate an outgoing connection (*connect*) or to wait for an incoming connection (*listen*).

Newton Internet Enabler

4. Connect the endpoint. If you are using a TCP link, pass down the TCP remote socket option.
5. Set up the input spec for your endpoint.
6. Set up the output spec for your endpoint.
7. Send and receive data.
8. Disconnect and dispose of your endpoint.
9. Release the link by calling the `InetReleaseLink` function.

Grabbing a Link

To get started, you need to establish (grab) a link. To establish a link, you need to call the `InetGrabLink` function. Before calling `InetGrabLink`, you should call the `InetOpenConnectionSlip` function to allow the user to modify the default link settings. This function also resets the default link ID.

`InetOpenConnectionSlip(linkID, clientContext, clientSlipCallback)`

After `InetOpenConnectionSlip` finishes, it calls your callback function to let you know whether or not to proceed with the connection process.

`InetOpenConnectionSlip` passes one parameter, a symbol, to your callback function. If the symbol is `'connect'`, you should proceed with the connection; if not, the user cancelled the connection,

After calling `InetOpenConnectionSlip`, you call `InetGrabLink` with a link ID, a callback function, and a callback context frame:

`InetGrabLink(linkID, clientContext, clientCallback)`

For the `linkID`, you can tell `InetGrabLink` to use the default link by using `nil` or you can use an identifier returned by the `InetAddNewLinkEntry` function as the value of this parameter. You can also call the `InetGetDefaultLinkID` function to get the ID of the default link. When you specify `nil`, the system software uses the link ID that has been established as the default link ID. This is the ID established by the user in the connection slip. You almost always use `nil` as the value of this parameter to

Newton Internet Enabler

specify the default link ID, especially if you have first called `InetOpenConnectionSlip`.

The `InetGrabLink` operation can take some time to complete. After you call the `InetGrabLink` function, the Newton system software repeatedly calls your callback function to report the current status of grabbing the link. `InetGrabLink` calls your callback function until either an error occurs or until the status becomes 'connected.

The status value in your callback is a status frame, as described in the section “The Link Controller Status Frame” beginning on page 1-50. This frame contains the current link status value and (possibly) other information. In your callback, you can use the `InetDisplayStatus` function to show the current status to the user. The next section, “Retrieving and Displaying Link Status Information,” describes how to display status to the user.

Here is an example of a callback function for the `InetOpenConnectionSlip` function:

```
mySlipCallback := func(action)
    if action = 'connect then
        InetGrabLink(nil, self, 'GrablinkCallback);
```

Here is an example of a callback function for the `InetGrabLink` function:

```
myApp.GrabLinkCallback := func(linkID, stat, err)
    begin
        myLinkID := linkID;
        if err then
            begin
                // handle the error
                GetRoot():Notify(kNotifyAlert, kAppName,
                                call kGetInetError with (err));
                // close the status dialog
                InetDisplayStatus(nil, myStatusView, nil);
            end
        else if status.linkStatus <> 'connected then
```

Newton Internet Enabler

```

begin
    // just update the status dialog
    InetDisplayStatus(nil, myStatusView, status);
end
else begin
    // now connected, soclose status dialog
    // or start sending your own status
    local myStatus := {
        statusText:"Resolving Internet Address",};
    InetDisplayStatus(nil, myStatusView, myStatus);
    // resolve our host name address
    DNSGetAddressFromName("apple.com", self,
                          DNSCallback);
end;
end;

```

The first statement, `myLinkID:=linkID`, saves the ID of the link that `InetGrabLink` is in the process of grabbing in one of your variables. You might want to store the link ID for use in other portions of your application.

If grabbing of the link is progressing without errors, your callback function gets called to report the progress. You can call the `InetDisplayStatus` function, as shown in the above example. The `myStatView` view used in the this example was created before the grab of the link was initiated, as shown in the next section, “Retrieving and Displaying Link Status Information.”

The grab of the link terminates when the connection is made or when an error occurs. In either case, you can remove the status display view at that point. To do so, call the `InetDisplayStatus` function with `nil` as the value of the status parameter.

If `InetGrabLink` encounters an error, the error code will be a non-zero value and your application has to do something with that error. In the example function, a message is displayed and the connection attempt is terminated.

If `InetGrabLink` succeeds, the callback receives 'connected' as the value of `linkStatus`. At that point, you can perform any operations that are

Newton Internet Enabler

appropriate. The example function takes this opportunity to convert its remote echo host name into an IP address, which is saved in a local variable by the `DNSCallback` function. While the name resolution is taking place, the example updates the status display with its own message.

Retrieving and Displaying Link Status Information

Many applications want to display status to the user while a net connection is being established. Newton Internet Enabler makes this easy for you with the `InetDisplayStatus` function, which displays link status information on the Newton screen. Here is the declaration of the function:

```
statusView InetDisplayStatus(linkID, statusView, status)
```

You can use the `InetDisplayStatus` function in three ways, as follows:

- to create a new status view, pass `nil` as the value of each parameter:

```
myStatusView := InetDisplayStatus(nil, nil, nil);
```
- to display status for a link in an existing status view, pass in the link ID, the status view, and the status frame that was sent to your callback function:

```
InetDisplayStatus(myLinkID, myStatusView, myStatus);
```
- to remove and dispose of the status view, pass `nil` as the value of the status frame:

```
InetDisplayStatus(myLinkID, myStatusView, nil);
```

The `InetStatusDisplay` function creates and uses a view that is based on `protoStatusTemplate`. For information about this proto, see the chapter “Additional System Services” in *Newton Programmer’s Guide*.

To initiate the status display, you need to open the status view. The most convenient place to do this is just before your call to the `InetGrabLink` function. For example, the following function creates the status view, stores it in `myStatView` for subsequent use, and then calls the `InetGrabLink` function:

Newton Internet Enabler

```

DoGrabLink := func()
begin
  myStatView := InetDisplayStatus(nil, nil, nil);
  InetGrabLink(nil, self, 'GrabLinkCallback);
end;

```

While the grab operation is in progress, you can update the status display whenever your callback function gets called. For example, the following code segment from a grab link callback function updates the status display if no errors have occurred and if the link status has not yet become 'connected:

```

if err = nil and status.linkStatus <> 'connected then
  InetDisplayStatus(linkID, myStatView, stat);

```

When the grab operation is done, you can remove the status display. The following code segment from a grab link callback function removes the status display when the link status becomes 'connected:

```

if err = nil and status.linkStatus = 'connected then
  InetDisplayStatus(linkID, myStatView, nil);

```

The view displayed by the `InetDisplayStatus` function contains a button that the user can tap to call the `InetCancelLink` function.

Configuring Newton Internet Enabler for Your Endpoint

After grabbing your Newton Internet Enabler link, you need to instantiate your endpoint. You send the `Instantiate` message to your endpoint with the options required to configure Newton Internet Enabler for your application.

You must set three options in your `Instantiate` message:

- The `'inet` service identifier option, which tells the Newton system software to use Newton Internet Enabler with your endpoint.

Newton Internet Enabler

- The Inet tool physical link ('`ilid`') option, which tells Newton Internet Enabler which link ID to use for your endpoint. Use the link ID that was returned by the `InetGrabLink` function.
- The Inet tool transport service type ('`itsv`') option, which tells Newton Internet Enabler which transport type (for example, UDP or TCP) to use for your endpoint.

Binding Your Endpoint with Newton Internet Enabler

After you instantiate your endpoint, you need to bind it to an address. You either bind your endpoint to *connect* (initiate an outgoing connection), or to *listen* for an incoming connection. If you are binding an endpoint that is going to listen, you always need to pass the Inet local port ('`ilpt`') option when you send the `Bind` message to your endpoint. If you are binding an endpoint that is going to connect, you need to pass the Inet local port option for UDP links, but not for TCP links.

The Inet local port option has two data slots that you specify: a short value, `InetPortNumber`, and a Boolean value, `useDefaultPort`. The `useDefaultPort` value only applies when you are binding an endpoint to

Newton Internet Enabler

connect over a UDP link. Assign the `InetPortNumber` a value as shown in Table 1-1 when sending the local port option with a `Bind` request:

Table 1-1 Local port numbers for binding with Newton Internet Enabler

Bind type	Transport service type	Local port number
For connect	TCP	The system always selects the local port number, so don't set this option. You can, however, send a <code>get (opGetCurrent)</code> of this option with your <code>Bind</code> to retrieve the port number that the system assigned.
For connect	UDP	If you specify <code>true</code> for <code>useDefaultPort</code> , Newton Internet Enabler will select the local port to use and will return its value in the option. If you specify <code>nil</code> for <code>useDefaultPort</code> , you must supply a port number that is not in use or the <code>Bind</code> will fail.
For listen	TCP	Specify a port number to listen on as defined by the <i>IETF Assigned Numbers RFC</i> document.
For listen	UDP	Specify a port number to listen on as defined by the <i>IETF Assigned Numbers RFC</i> document.

Connecting Your Endpoint with Newton Internet Enabler

After instantiating and binding your endpoint, you need to connect it. If you are using a TCP link, you need to pass the TCP remote socket (`'itrs'`) option when you send the `Connect` message to your endpoint. This option sets the host address with which TCP connects. You can use the domain name server to get this address, as described in the section “Using the Domain Name Service Interface” beginning on page 1-28.

Newton Internet Enabler

If you are using a UDP link, you do not need to pass any options in your `Connect` message.

If you are sending the `Listen` message to your endpoint, you do not need to send any options with that message.

Sending Data

You use Newton Internet Enabler to send data just as you would with any Newton communications tool. You can set up an output specification frame and send the `Output` message to your endpoint after you have established a connection.

Note

When you are sending data over the Internet, you usually need to insert a linefeed (`unicodeLF`) character in your data. Most Internet data uses linefeed-carriage return pairs, while the Newton uses only carriage returns. ♦

Sending Data With a UDP Connection

For UDP connections, you need to include the `Inet UDP destination socket ('iuds')` option to establish the destination of the UDP datagram. Your UDP output specification must include two flags in the `sendFlags` slot: the `kPacket` and `kEOP` flags.

WARNING

When you send data with a UDP connection, the output specification must not be `nil`. ▲

The following code segment sends the string “Hello World!” out over a UDP link.

```
local myUDPstreamOutputSpec := {
    form:          'string,
    sendFlags:     'kPacket+'kEOP,
}
```

Newton Internet Enabler

```

local myUDPOptions :=
[
  {
    label:      "iuds",
    type:       'option,
    opCode:     opSetCurrent,
    result:     nil,
    form:       'template,
    data:
    {
      arglist:
      [
        [130,43,2,2], // host address
        7,             // destination port number
      ],
      typelist:
      [
        'struct,
        ['array, 'byte, 4],
        'short
      ]
    }
  }
];

try
  ep:Output("Hello World!", myUDPOptions,
            myUDPstreamOutputSpec);
onexception |evt.ex.comm| do
  return :DoDisconnect();

```

Sending Data With a TCP Connection

For TCP links, you do not need to include any options in your **Output** message, nor do you need to specify any **sendFlags** values in the output

Newton Internet Enabler

specification frame. For example, the following code segment sends the string “Hello World!” out over a TCP link.

```
local myTCPstreamOutputSpec := {
    form:          'string',
}

try
    ep:Output("Hello World!", nil, myTCPstreamOutputSpec);
onexception |evt.ex.comm| do
    return :DoDisconnect();
```

The above example calls the application’s `DoDisconnect` function if any communication exception occurs while sending the data.

You can also send **expedited data** over a TCP link. Expedited data is a single byte of data that gets sent immediately. The data byte gets inserted in front of any data on the remote end that has been received but not yet processed. For example, you might need to send out a break character in the middle of transmitting a large amount of data. To do so, you use the `Inet expedited` data option with your `Output` message. The expedited data option is described on page 1-73.

See the chapter “Endpoint Interface” in *Newton Programmer’s Guide* for detailed information about output specification frames and the `Output` method.

Receiving Data

You use Newton Internet Enabler to receive data just as you would with any Newton communications tool. Typically, this means that you set up an input specification frame and send the `SetInputSpec` message to your endpoint.

Newton Internet Enabler

Note

When you are receiving data from the Internet, you usually need to strip the linefeed (`unicodeLF`) characters from your data. Most Internet data uses linefeed-carriage return pairs, while the Newton just uses the carriage return. ♦

Receiving Data With UDP

For UDP links, your input specification frame must include the `kPacket` receive flag and must include `useEOP: true` in the termination slot. In addition, you can include two options in the `rcvOptions` slot if you want to: include the UDP source socket option to retrieve the address of the datagram sender, and include the UDP destination socket option if you want to retrieve the exact address to which the packet you received was sent. The destination address might be other than your local address if the packet was sent to a broadcast address.

The following code segment receives a datagram packet over a UDP link.

```
local streamInputSpec := {
  form:          'string,
  termination:   {useEOP: true},
  discardAfter:  565,
  rcvFlags:      kPacket,
  rcvOptions:    {
    label:       "iuss",
    type:        'option,
    opCode:      opGetCurrent,
    result:      nil,
    form:        'template,
    data: {
      arglist:
      [
        [0,0,0,0], // host address
        0, // host port number
      ],
    },
  },
}
```

```

                                typelist: kPortAddrStruct,
                                [
                                    'struct,
                                    ['array, 'byte, 4],
                                    'short
                                ]
                            },
                        },
                    },
                },
            },
        },
    },
}

inputScript: func(ep, data, terminator, options)
begin
    // do something with data
end,

completionScript: func(ep, options, result)
begin
    // skip error handling for cancelled requests
    if result <> kCommAbortErr then
        begin
            print("Error: " && result);
            ep:DoDisconnect();
        end;
    end,
}

try
    ep:SetInputSpec(streamInputSpec);
onexception |evt.ex.comm| do
    return :DoDisconnect();

```

The example input specification frame above tells Newton Internet Enabler to receive a packet of data from the UDP link and provides two scripts: the `inputScript` function to process normal completion of data reception and the `completionScript` function to process unexpected termination of data

Newton Internet Enabler

reception. In addition, this input spec includes a “get” of the UDP source socket address, which will be filled in with the IP address of the host that sent the datagram to your application.

WARNING

If you attempt to send or receive a packet larger than the value specified in the `discardAfter` slot of your input specification frame, a `kCommErrBufferOverflow` error results and some data may be lost. The maximum size of a UDP packet for Newton Internet Enabler is 565. ▲

Receiving Data With TCP

For TCP links, you do not need to include any options or specify any receive flags in your input specification frame. For example, the following code segment receives a carriage return-terminated string from a TCP connection.

```
local streamInputSpec := {
    form:          'string,
    termination:   {endSequence: UnicodeCR},

    inputScript: func(ep, data, terminator, options)
        begin
            // do something with data
        end,

    completionScript: func(ep, options, result)
        begin
            // skip error handling for cancelled requests
            if result <> kCommAbortErr then
                begin
                    print("Error: " && result);
                    ep:DoDisconnect();
                end;
            end,
        end,
```

Newton Internet Enabler

```
}

try
    ep:SetInputSpec(streamInputSpec);
onexception |evt.ex.comm| do
    return :DoDisconnect();
```

The example input specification frame above tells Newton Internet Enabler to terminate input upon receiving a Unicode carriage return character and provides two scripts: the `inputScript` function to process normal completion of data reception and the `completionScript` function to process unexpected termination of data reception.

WARNING

Do not use the `discardAfter` slot in your input specification for TCP connections. If you want to limit the size of the data packet, specify the `byteCount` value in your termination frame. ▲

You can also receive expedited data over a TCP link. When expedited data arrives, your application is immediately notified: the link controller sends an application event frame. The `eventCode` slot of this event frame has the value `kEventToolSpecific` and the `data` slot is the byte that was received.

See the chapter “Endpoint Interface” in *Newton Programmer’s Guide* for detailed information about input specifications, the `SetInputSpec` method, handling communications events, and other styles of receiving data with an endpoint.

Disconnecting Your Endpoint

When you have finished using your endpoint, you need to disconnect, unbind, and dispose of it. The following function shows you an example of finishing your use of an endpoint.

Newton Internet Enabler

```

MyApp.DoDisconnect := func()
begin
  if ep then begin// ignore all disconnect errors
    try
      ep:Disconnect(true, nil);
    onexception |evt.ex.comm| do
      nil;

    try
      ep:UnBind(nil)
    onexception |evt.ex.comm| do
      nil;

    try
      ep:Dispose()
    onexception |evt.ex.comm| do
      nil;
    end;
  end;
end;

```

Releasing Your Link

After your application is completely done with the link, or whenever you will not be using the link for a long period of time (approximately 15 minutes or longer), you need to release it by calling the `InetReleaseLink` function. If no other applications are using the link, the Newton system software shuts it down.

You need to provide `InetReleaseLink` with a link ID, a callback function, and a callback context frame:

Newton Internet Enabler

`InetReleaseLink(linkID, clientContext, clientCallback)`

IMPORTANT

The *clientContext* value that you specify in your `InetReleaseLink` call must match the *clientContext* value that you previously specified in your call to the `InetGrabLink` function for this link. Otherwise, an error will occur. ▲

The status value in your callback is a status frame, as described in the section “The Link Controller Status Frame” beginning on page 1-50. This frame contains the current link status value and other information. In your callback, you can determine the current status of the link after your release of it has completed, which will depend upon its use by other applications.

Here is an example of a callback function for the `InetReleaseLink` function:

```
myApp.ReleaseLinkCallback := func(linkID, stat, err)
begin
  if stat.linkStatus = 'idle then
    ; // do something here
  end;
```

Power Management and Internet Links

Whenever a link is active or a grablink is in progress, the Newton Internet Enabler modifies the handling of power-down requests in the following two ways:

- The standard “power-down when idle” feature of the Newton is disabled.
- If the user tries to power down with the power switch, the Newton Internet Enabler displays a dialog asking the user if he or she really wants to drop the link. If the user taps OK, the link is dropped and the Newton is powered down. If the user taps Cancel, the link is retained and the Newton remains powered.

Using the Domain Name Service Interface

You can use the Newton Internet Enabler domain name service functions to translate between host name and Internet address representations. Newton Internet Enabler provides the following domain name service global functions:

- the `DNSCancelRequests` function cancels any pending DNS requests.
- the `DNSGetAddressFromName` function translates a domain name into its corresponding Internet address.
- the `DNSGetMailAddress` function translates a domain name into the Internet address for a mail server that serves that domain.
- the `DNSGetMailServerNameFromDomainName` function translates a domain name into the domain name for a mail server that serves that domain.
- the `DSNGetNameFromAddress` function translates an Internet address into its corresponding domain name.

Each of the DNS global functions is described in the section “Domain Name Service Functions and Methods” beginning on page 1-67.

You must supply a *clientContext* and *clientCallback* parameter to each of the DNS functions, just as you do for the link controller functions. However, the DNS callback functions are called with different parameters than are the link controller functions.

The callback function for `DNSCancelRequests` receives no parameters.

The callback function for all of the other DNS functions receives two parameters: an array of DNS results frames and a result code. Each results frame contains a number of slots that describe the DNS operation that was performed. The format of these parameters is described in the section “The Domain Name Service Callback Function Format” beginning on page 1-55.

For example, the `DNSGetAddressFromName` function is declared as follows:

```
DNSGetAddressFromName(addr, clientContext, clientCallback)
```

An example of a callback for this function is shown here:

Newton Internet Enabler

```
myApp.DNSGetAddrcallback := func(results, error)
begin
  if error or length(results) < 1 then
    begin
      print("DNS error: " && error);
      // do something with the error
      return;
    end;

    // save the resolved address
    myRemoteIpAddr := results[0].resultIPAddress;
  end;
```

Using the Newton Internet Enabler Options

Table 1-2 describes the Newton Internet Enabler options. Each of these options is described in detail in the section “Newton Internet Enabler Options” beginning on page 1-73.

Table 1-2 Newton Internet Enabler options

Option name	Description	When to use
Expedited data transfer ('iexp')	For expedited transmission of data over a TCP link.	Set this option with an Output call to transfer data on a TCP endpoint.
Physical link identifier ('ilid')	To identify the link ID to use.	Set this option at endpoint instantiation time.
Local port ('ilpt')	To set the local port number for TCP binds.	Set this option if you are binding to do a Listen (at endpoint instantiation or bind time). You don't need to set this option for a Connect .

Newton Internet Enabler

Table 1-2 Newton Internet Enabler options (continued)

Option name	Description	When to use
Inet profile (<code>'iprf'</code>)	To set the local port number for UDP binds.	Set this option at endpoint instantiation or bind time.
	To retrieve the local port number used for TCP or UDP.	Retrieve the value of this option when you are connecting, sending, or receiving data.
	To retrieve the local host and gateway host IP addresses.	At any time after the link is established.
TCP remote socket (<code>'itrs'</code>)	To set the socket to which TCP connects.	Set the value of this option at before using the connection (at endpoint instantiation, bind, or connect time).
	To retrieve the sender address for data received over a TCP link.	Get the value of this option when listening for data on a TCP connection.
Transport service type (<code>'itsv'</code>)	To set the transport service type (TCP or UDP).	Set this option at endpoint instantiation time.
UDP destination socket (<code>'iuds'</code>)	To set the destination address for data being sent over a UDP link.	Set this option when sending data with a UDP connection.
	To retrieve the destination address for data received over a UDP link.	Get this option when listening for data on a UDP connection.
UDP source socket (<code>'iuuss'</code>)	To retrieve the source address for data received over a UDP link.	Get the value of this option when listening for data on a UDP connection.

Newton Internet Enabler Interface Reference

This section describes the constants, data types, methods, and functions that you use with Newton Internet Enabler.

Constants

This section describes the constants that you use with Newton Internet Enabler.

Status Code Constants

The Newton system software uses the following status code symbols to convey the status of a Newton Internet Enabler operation:

'idle
'initializing
'connecting
'login
'connected

Constant descriptions

'idle	No link is established.
'initializing	Setting up for connecting.
'connecting	In the process of connecting.
'login	Performing login script.
'connected	The link is established.

Transport Service Type Constants

You can use the following constants to specify transport service types in Inet tool option requests:

Newton Internet Enabler

```
constant kTCP    := 1;
constant kUDP    := 2;
```

Constant descriptions

kTCP	Use TCP transport service.
kUDP	Use UDP transport service.

Link Controller Error Codes

The high-level link controller can generate the errors described in this section.

```
constant kInetErrNoSuchLinkID      := -60501;
constant kInetErrLinkDisconnected  := -60504;
constant kInetErrConnectLinkFailed := -60505;
```

Constant descriptions

kInetErrNoSuchLinkID	The specified link identifier does not exist.
kInetErrnLinkDisconnected	The link has been disconnected.
kInetErrConnectLinkFailed	The link could not connect.

DNS Error Codes

This section describes the error codes that the DNS tool can generate.

```
constant kDNSErrNoAnswerFoundYet      -60751;
constant kDNSErrInternalErr           -60752;
constant kDNSErrNameSyntaxErr         -60791;
constant kDNSErrNoNameServer          -60794;
constant kDNSErrAuthNameErr           -60795;
constant kDNSErrNoAnswerErr           -60796;
constant kDNSErrNonexistentDomain     -60797;
constant kDNSErrOutOfMemory           -60798;
```

Newton Internet Enabler

```
constant kDNSErrCouldNotContactServer    -60800;  
constant kDNSErrNoServersAvailable       -60801;  
constant kDNSErrRequestFormatErr        -60802;  
constant kDNSErrServerInternalErr       -60803;  
constant kDNSErrServerNotImplemented    -60804;  
constant kDNSErrServerRefused           -60805;  
constant kDNSErrUnknownServerError      -60806;  
constant kDNSErrNoResponseFromServer    -60814;  
constant kDNSErrNoResponseFromAnyServer -60815;
```

Newton Internet Enabler

Constant descriptions**kDNSErrNoAnswerFoundYet**

The answer for the question has not yet been found.

kDNSErrInternalErr

Internal DNS tool error.

kDNSErrNameSyntaxErr

The name in the DNS request is not valid.

kDNSErrNoNameServer

The option specification does not contain a name server.

kDNSErrAuthNameErr

The domain does not exist.

kDNSErrNoAnswerErr

No answers available for request; this could be due to a domain that does not exist.

kDNSErrNonexistentDomain

The domain name does not exist.

kDNSErrOutOfMemory

DNS tool out of memory.

kDNSErrCouldNotContactServer

Could not connect to the current DNS server

kDNSErrNoServersAvailable

Could not connect to any of the listed DNS servers.

kDNSErrRequestFormatErr

The DNS server did not like the format of the request, which could indicate an invalid domain name.

kDNSErrServerInternalErr

An internal error occurred in the DNS server.

kDNSErrServerNotImplemented

The DNS server does not support the specified type of request.

kDNSErrServerRefused

The DNS server refused to answer the client's query.

kDNSErrUnknownServerError

The DNS server returned an error code that is not recognized.

Newton Internet Enabler

`kDNSErrNoResponseFromServer`

No response from the current server

`kDNSErrNoResponseFromAnyServer`

No response from any of the available DNS servers.

Newton Internet Enabler PPP Errors

This section describes the PPP errors.

`constant kInetToolErrPppAuthSetupFailed` -60084;

`constant kInetToolErrPppAuthReqFailed` -60085;

Constant descriptions

`kInetToolErrPppAuthSetupFailed`

The authentication type was rejected, probably because the remote host does not support PAP or CHAP.

`kInetToolErrPppAuthReqFailed`

The authentication request failed, probably due to incorrect user name and/or password entry.

Newton Internet Enabler Lower-Level Tool Errors

The Newton Internet Enabler lower-level errors are separated into five sections:

- Inet tool errors
- application-related errors
- UDP errors
- Inet tool-specific errors
- PPP internal errors

Inet Tool Errors

This section describes the errors that the built-in Inet tool can generate.

Newton Internet Enabler

Note

Some of the errors in this section are internal communications stack configuration or operation errors. If you receive one of these errors, please contact the Newton Developer Technical Support team. ♦

constant kInetToolErrBindFailed	:= -60001;
constant kInetToolErrIPBindFailed	:= -60002;
constant kInetToolErrPushModule	:= -60004;
constant kInetToolErrIlink	:= -60005;
constant kInetToolErrNetActivateReq	:= -60006;
constant kInetToolErrTCPBind	:= -60007;
constant kInetToolErrGetRequest	:= -60008;
constant kInetToolErrPutRequest	:= -60009;
constant kInetToolErrConnect	:= -60010;
constant kInetToolErrDlAttach	:= -60011;
constant kInetToolErrBind	:= -60012;
constant kInetToolErrOpenLink	:= -60013;
constant kInetToolErrUnlink	:= -60014;
constant kInetToolErrOutOfPhase	:= -60015;
constant kInetToolErrAddRoute	:= -60016;
constant kInetToolErrListen	:= -60017;
constant kInetToolErrLinkNotOpened	:= -60018;
constant kInetToolErrDriverNotOpened	:= -60019;
constant kInetToolErrStreamNotOpened	:= -60020;
constant kInetToolErrBindReqFailed	:= -60021;
constant kInetToolErrConnResReqFailed	:= -60022;

Newton Internet Enabler

Constant descriptions**kInetToolErrBindFailed**

The bind operation failed at the lowest level.

kInetToolErrIPBindFailed

The IP layer bind operation failed.

kInetToolErrPushModule

Internal communications stack configuration error.

kInetToolErrILink

Internal communications stack configuration error.

kInetToolErrNetActivateReq

Internal communications stack configuration error.

kInetToolErrTCPBind

The TCP layer bind operation failed.

kInetToolErrGetRequest

The get request resulted in an error.

kInetToolErrPutRequest

The put request resulted in an error.

kInetToolErrConnect

The connect request resulted in an error.

kInetToolErrDLAttach

Internal communications stack configuration error.

kInetToolErrBind

Internal communications stack configuration error.

kInetToolErrOpenLink

Internal communications stack configuration error.

kInetToolErrUnlink

Internal communications stack configuration error.

kInetToolErrOutOfPhase

Internal communications stack configuration error (the stack layers are out of sync).

kInetToolErrAddRoute

Internal communications stack configuration error.

kInetToolErrListen

Internal communications stack configuration error.

Newton Internet Enabler

<code>kInetToolErrLinkNotOpened</code>	Internal communications stack configuration error.
<code>kInetToolErrDriverNotOpened</code>	Internal communications stack configuration error.
<code>kInetToolErrStreamNotOpened</code>	Internal communications stack configuration error.
<code>kInetToolErrBindReqFailed</code>	The bind request failed.
<code>kInetToolErrConnResReqFailed</code>	Internal communications stack configuration error.

Application-related Errors

This section describes the Inet errors that are related to your application.

Note

Some of the errors in this section are internal communications stack configuration or operation errors. If you receive one of these errors, please contact the Newton Developer Technical Support team. ♦

<code>constant kInetToolErrMemAlloc</code>	<code>:= -60023;</code>
<code>constant kInetToolErrMsgType</code>	<code>:= -60024;</code>
<code>constant kInetToolErrNoDevice</code>	<code>:= -60025;</code>
<code>constant kInetToolErrIllegalOpenOnStream</code>	<code>:= -60026;</code>
<code>constant kInetToolErrReqInInvalidState</code>	<code>:= -60027;</code>
<code>constant kInetToolErrPrimitiveTooSmall</code>	<code>:= -60028;</code>
<code>constant kInetToolErrPrimitiveOutOfRange</code>	<code>:= -60029;</code>
<code>constant kInetToolErrPrimitiveOnInvalidStr</code>	<code>:= -60030;</code>
<code>constant kInetToolErrMessageTooLong</code>	<code>:= -60031;</code>
<code>constant kInetToolErrNetworkAlreadyActive</code>	<code>:= -60032;</code>
<code>constant kInetToolErrNetworkNumberInvalid</code>	<code>:= -60033;</code>
<code>constant kInetToolErrUnsupportedIoctl</code>	<code>:= -60034;</code>
<code>constant kInetToolErrStreamAlreadyAttached</code>	<code>:= -60035;</code>
<code>constant kInetToolErrUnknownMuxIndex</code>	<code>:= -60036;</code>

Newton Internet Enabler

```
constant kInetToolErrNetworkIsInactive      := -60037;
constant kInetToolErrBogusConnection        := -60038;
constant kInetToolErrInvalidBillingMode     := -60039;
constant kInetToolErrNoTrigSelectedInAlarm  := -60040;
constant kInetToolErrInvalidTrigSize        := -60041;
constant kInetToolErrInvalidConnectionRef   := -60042;
constant kInetToolErrIllegalMdataInPrim     := -60043;
constant kInetToolErrMissingMdataInPrim     := -60044;
constant kInetToolErrInvalidSegmentedPrim   := -60045;
constant kInetToolErrInvalidNPIVersion      := -60046;
constant kInetToolErrInvalidAddress         := -60047;
constant kInetToolErrOutOfTCPPortNumbers    := -60048;
constant kInetToolErrSocketInUse            := -60049;
constant kInetToolErrReservedPortNumber     := -60050;
constant kInetToolErrExpDataNotSupported    := -60051;
```

Newton Internet Enabler

Constant descriptions**kInetToolErrMemAlloc**

Requested memory could not be allocated.

kInetToolErrMsgType

Internal communications stack configuration error.

kInetToolErrNoDevice

Internal communications stack configuration error: no stream available.

kInetToolErrIllegalOpenOnStream

Internal communications stack configuration error.

kInetToolErrReqInInvalidState

Internal communications stack configuration error.

kInetToolErrPrimitiveTooSmall

Internal communications stack configuration error.

kInetToolErrPrimitiveOutOfRange)

Internal communications stack configuration error.

kInetToolErrPrimitiveOnInvalidStr

Internal communications stack configuration error.

kInetToolErrMessageTooLong

Internal communications stack configuration error.

kInetToolErrNetworkAlreadyActive

Internal communications stack configuration error.

kInetToolErrNetworkNumberInvalid

Internal communications stack configuration error.

kInetToolErrUnsupportedIoctl

Internal communications stack configuration error.

kInetToolErrStreamAlreadyAttached

Internal communications stack configuration error.

kInetToolErrUnknownMuxIndex

Internal communications stack configuration error.

kInetToolErrNetworkIsInactive

Internal communications stack configuration error.

kInetToolErrBogusConnection

Internal communications stack configuration error.

Newton Internet Enabler

<code>kInetToolErrInvalidBillingMode</code>	Internal communications stack configuration error.
<code>kInetToolErrNoTrigSelectedInAlarm</code>	Internal communications stack configuration error.
<code>kInetToolErrInvalidTrigSize</code>	Internal communications stack configuration error.
<code>kInetToolErrInvalidConnectionRef</code>	Internal communications stack configuration error.
<code>kInetToolErrIllegalMdataInPrim</code>	Internal communications stack configuration error.
<code>kInetToolErrMissingMdataInPrim</code>	Internal communications stack configuration error.
<code>kInetToolErrInvalidSegmentedPrim</code>	Internal communications stack configuration error.
<code>kInetToolErrInvalidNPVersion</code>	Internal communications stack configuration error.
<code>kInetToolErrInvalidAddress</code>	Internal communications stack configuration error.
<code>kInetToolErrOutOfTCPPortNumbers</code>	Internal communications stack configuration error.
<code>kInetToolErrSocketInUse</code>	Internal communications stack configuration error.
<code>kInetToolErrReservedPortNumber</code>	Internal communications stack configuration error.
<code>kInetToolErrExpDataNotSupported</code>	Internal communications stack configuration error.

UDP Errors

This section describes the UDP-related errors that the built-in Inet tool can generate.

Newton Internet Enabler

Note

Some of the errors in this section are internal communications stack configuration or operation errors. If you receive one of these errors, please contact the Newton Developer Technical Support team. ♦

```
constant kInetToolErrRedundentRequest      := -60052;
constant kInetToolErrUnexpectedDLPrim     := -60053;
constant kInetToolErrUnexpectedTPIPrim    := -60054;
constant kInetToolErrUnexpectedNPIPrim    := -60055;
constant kInetToolErrUnknownTPIErrorCode := -60056;
```

Constant descriptions

```
kInetToolErrRedundentRequest
    Internal communications stack configuration error.
kInetToolErrUnexpectedDLPrim
    Internal communications stack configuration error.
kInetToolErrUnexpectedTPIPrim
    Internal communications stack configuration error.
kInetToolErrUnexpectedNPIPrim
    Internal communications stack configuration error.
kInetToolErrUnknownTPIErrorCode
    Internal communications stack configuration error.
```

Inet Tool-Specific Errors

This section describes the errors that relate to the Inet tool disconnecting unexpectedly.

```
constant kInetErrStreamInoperative      := -60057;
constant kInetErrUnexpectedMgmtEvent    := -60058;
```

Newton Internet Enabler

Constant descriptions**kInetErrStreamInoperative**

The communications connection shut down due to a fatal error.

kInetErrUnexpectedMgmtEvent

The communications connection shut down due to an internal error.

PPP Internal Errors

This section describes the internal PPP errors.

Note

Some of the errors in this section are internal communications stack configuration or operation errors. If you receive one of these errors, please contact the Newton Developer Technical Support team. ♦

```
constant kInetErrLinkActivationNotConfirmed := -60059;
constant kInetErrLinkDeactivationNotConfirmed := -60060;
constant kInetErrLinkTerminateAckNotReceived := -60061;
constant kInetErrNetworkDifferentFromTheReq := -60062;
constant kInetErrConnLossFromTerminateReq := -60063;
constant kInetErrConnLossFromConfigureRequest := -60064;
constant kInetErrConnLossFromConfigureReject := -60065;
constant kInetErrConnLossFromConfigureAck := -60066;
constant kInetErrConnLossFromConfigureNack := -60067;
constant kInetErrConnLossFromCodeReject := -60068;
constant kInetErrConnLossFromMaxSetUpAttempts := -60069;
constant kInetErrConnLossFromMandatoryFrameSize := -60070;
constant kInetErrConnLossFromMandatoryAuthRej := -60071;
constant kInetErrConnLossFromRemoteInitAuthFail := -60072;
constant kInetErrConnLossFromLocalInitAuthFail := -60073;
constant kInetErrConnLossFromLowerLinkFailure := -60074;
constant kInetErrConnLossUnspecifiedReason := -60075;
constant kInetErrConnLossFromMaxRemoteInitAuthFail
```

Newton Internet Enabler

```
                                := -60076;  
constant kInetErrConnLossFromMaxLocalInitAuthFail  
                                := -60077;  
constant kInetErrConnLossFromMngmntDeactivation:= -60078;  
constant kInetErrConnLossFromNoIpAddr         := -60079;  
  
constant kInetToolErrDataLinkLevel            := -60080;  
constant kInetToolErrSubsBind                 := -60081;  
constant kInetToolErrNetDeActivateReq        := -60082;  
constant kInetToolErrNetworkLevel            := -60083;
```


Newton Internet Enabler

Constant descriptions**kInetErrLinkActivationNotConfirmed**

Link activation not confirmed.

kInetErrLinkDeactivationNotConfirmed

Link deactivation not confirmed.

kInetErrLinkTerminateAckNotReceived

Termination acknowledgment not received.

kInetErrNetworkDifferentFromTheReq

Illegal value for internal network parameter.

kInetErrConnLossFromTerminateReq

The connection was lost due to a terminate request.

kInetErrConnLossFromConfigureRequest

The connection was lost due to a configuration request.

kInetErrConnLossFromConfigureReject

The connection was lost due to a configuration rejection.

kInetErrConnLossFromConfigureAck

The connection was lost due to a configuration acknowledgment.

kInetErrConnLossFromConfigureNack

The connection was lost due to a configuration NACK.

kInetErrConnLossFromCodeReject

The connection was lost due to a code rejection.

kInetErrConnLossFromMaxSetUpAttempts

The connection was lost due to exceeding the maximum number of setup attempts.

kInetErrConnLossFromMandatoryFrameSize

The connection was lost due to not meeting the mandatory frame size.

kInetErrConnLossFromMandatoryAuthReq

The connection was lost due to the rejection of a mandatory authentication option configuration request.

kInetErrConnLossFromRemoteInitAuthFail

The connection was lost due to a failure of the authentication initiated by the remote host.

kInetErrConnLossFromLocalInitAuthFail

Newton Internet Enabler

The connection was lost due to a failure of the authentication initiated by the local host..

`kInetErrConnLossFromLowerLinkFailure`

The connection was lost due to a failure at a lower link level.

`kInetErrConnLossUnspecifiedReason`

The connection was lost for an unspecified reason.

`kInetErrConnLossFromMaxRemoteInitAuthFail`

The connection was lost due to the remote host exceeding the maximum number of authentication initiations.

`kInetErrConnLossFromMaxLocalInitAuthFail`

The connection was lost due to the local host exceeding the maximum number of authentication initiations.

`kInetErrConnLossFromMngmntDeactivation`

The connection was lost due to an internal deactivation request.

`kInetErrConnLossFromNoIpAddr`

The connection was lost because no valid IP address was negotiated.

`kInetToolErrDataLinkLevel`

A data link level error occurred.

`kInetToolErrSubsBind`

A subtool binding error occurred.

`kInetToolErrNetDeActivateReq`

An error occurred in the net deactivation request.

`kInetToolErrNetworkLevel`

A network level error occurred.

Newton Internet Enabler Events

The Newton Internet Enabler generates some events that you can handle in the `EventHandler` method of your endpoints. The `EventHandler` method, which is described in the chapter “Endpoint Interface” of *Newton*

Newton Internet Enabler

Programmer's Guide, receives a single parameter, which is an event frame. The slots of the event frame are as follows:

eventCode	An integer event code.
data	An integer representing event data.
serviceId	A string representing the communication tool that originated the event. For example, "mods" identifies the modem tool.
time	An integer representing the time when the event occurred. This is the number of ticks since the system was last restarted, not including time when it was turned off.

When the Newton Internet Enabler sends an event to your application, the `serviceId` is 'inet'. The event codes and data that Newton Internet Enabler can send are shown in Table 1-3.

Table 1-3 Newton Internet Enabler Application Events

Event code	Data value	Description
kEventToolSpecific	An error code; currently, this is always the value: <code>kInetErrStreamInoperative</code>	Sent to your endpoint when the connection becomes inoperative due to a fatal error.
kEventToolSpecific	A byte of data received for the application.	Sent to your endpoint when an expedited data byte arrives. This is only applicable to endpoints using TCP.
kCommToolEventDisconnected	none	Sent to your endpoint if the remote side disconnects a TCP connection.

Since the `eventCode` slot has the same value (`kEventToolSpecific`) in the first two cases shown in Table 1-3, you must check the value of the data slot to determine what has happened:

Newton Internet Enabler

- if the `data` slot value is less than zero, you know that your connection has closed down due to an error. The `data` slot value is the error code.
- otherwise, you know that your application has just received a byte of expedited data. The `data` slot value is the byte of expedited data that has just been received.

For more information on receiving expedited data, see the section “Receiving Data With TCP” beginning on page 1-24.

Handling TCP Disconnect Events

If the remote side disconnects a TCP connection, your endpoint generates a disconnect event. However, you might still need to process data that has been received and buffered prior to the disconnect. To process the buffered data, you need to post input specifications until you get the error `kCommErrNotConnected`.

Note

You must disconnect and unbind your endpoint event if you receive a disconnect (`kCommToolEventDisconnected`) event. ♦

Newton Internet Enabler Function Parameter Information

Many of the Newton Internet Enabler functions use one or more of the parameter types described here.

The Link Identifier Parameter

The link identifier parameter, which is used by a number of the link controller functions, defines the link that you want to use for a link controller operation. If you specify `nil` for the link identifier, Newton Internet Enabler substitutes the default link identifier.

Users define the default link ID to use in the Internet Setup application. Whenever you successfully call the `InetGrabLink` function, the link ID that you supplied to that function will automatically become the default link ID.

Newton Internet Enabler

Under most circumstances, you should specify `nil` as the value of the link identifier to use the default link ID as established by the user.

The Client Context Parameter

The client context parameter, which is used by a number of the link controller functions, is a reference to a frame. The referenced frame must contain the callback function that is defined in the client callback parameter.

Note

You must provide the same client context frame to some of the Newton Internet Enabler functions that are called in pairs. For example, the `InetCancelLink` function can only cancel a link if the *clientContext* parameter that you pass into `InetCancelLink` matches the *clientContext* parameter that you previously passed into `InetGrabLink`. ♦

The Client Callback Parameter

The client callback parameter, which is used by a number of the Newton Internet Enabler functions, is the symbol for a function. This function must be defined in the frame that is defined by the client context parameter.

In most cases, the Newton system software calls your callback function upon completion of its operations. In some cases, notably the `InetGrabLink` function, the Newton system software calls your callback function repeatedly during the operation to keep you informed of the status.

If you are calling a link controller function, the declaration of your callback function must match the specification described in the next section, “The Link Controller Callback Function Format.”

If you are calling a domain name service function, the declaration of your callback function must match the specification in the section “The Domain Name Service Callback Function Format” beginning on page 1-55.

Newton Internet Enabler

The Link Controller Callback Function Format

The callback function that you supply to each link controller call must be declared with the following format:

```
func callBackFcn(linkId, statusFrame, err);
```

The system software calls `callBackFcn` after the Newton Internet Enabler function has completed.

<i>statusFrame</i>	A frame with the format shown in the next section, “The Link Controller Status Frame.” The <i>statusFrame</i> provides information on the current state of the link.
<i>err</i>	The operation result. If the value is <code>nil</code> , the operation was successful; otherwise, the value is one of the error codes shown in the section “Newton Internet Enabler Lower-Level Tool Errors” on page 1-35.

The Link Controller Status Frame

The link controller status frame contains a slot that conveys link status information to you. This frame is designed to work closely with views based on the `protoStatusTemplate` view, which is described in the Transport Interface chapter of *Newton Programmer's Guide*.

Note that the status frame can have other slots in it; however, only the *linkStatus* slot is available for application use.

Slot descriptions

<i>linkStatus</i>	A symbol that describes the current status of the link. This is one of the values described in the section “Status Code Constants” on page 1-31.
-------------------	--

The following is an example of a link controller status frame:

```
{
    linkStatus: 'connected
}
```

The Link Entry Information Frame

The Newton Internet Enabler accesses link entry information that is stored in an internal soup. Each entry in this soup defines link configuration information and is represented by a link entry information frame. An example of a link entry information frame follows:

```
{
    linkID:                0,
    tags:                  ['Inet'],
    setupName:             "My dial-up",
    physicalLayer:         'modem',
    linkLayer:             'PPP',
    localAddress:          [205,149,167,179],
    localAddressFixed:     nil,
    gatewayAddress:        [204,156,128,1],
    gatewayAddressFixed:   nil,
    phoneNumber:           "(408) 555-1234",
    userName:              "",
    password:              "",
    dnsServerAddress:      [204,156,128,1],
    defaultDomain:         ".",
    loginInfo:             {interpreterSymbol: 'default,
                           loginScriptTimeout: 60,
                           loginInstructions: [
                               {type: 'waitFor, data: "ogin:"},
                               {type: 'userName},
                               {type: 'sendCR},
                               {type: 'waitFor, data: "word"},
                               {type: 'password},
                               {type: 'sendCR},
                               {type: 'waitFor, data:"ing..."},
                           ],
                           }
}
```

CHAPTER 1

Newton Internet Enabler

}

Newton Internet Enabler

Slot descriptions

<i>linkID</i>	The (integer) ID for this link. This ID is generated by the link controller.
<i>tags</i>	Reserved for internal use. Do not modify.
<i>setupName</i>	The name of this link setup entry.
<i>physicalLayer</i>	The symbol for which physical layer to use when connecting. Use either 'modem or 'serial.
<i>linkLayer</i>	The symbol for which link-layer protocol to use for this link. Use either 'PPP or 'SLIP.
<i>localAddress</i>	Optional. The manually-entered local IP address of the Newton.
<i>localAddressFixed</i>	True if a fixed local address is required, nil if not.
<i>gatewayAddress</i>	Optional. The IP address of the gateway host to which the Inet tool connects.
<i>gatewayAddressFixed</i>	True if a fixed host address is required, nil if not.
<i>phoneNumber</i>	Optional. The phone number for the modem to dial.
<i>userName</i>	Optional. The user name to use as the account name for login scripts.
<i>passWord</i>	Optional. The password to use as the account password for login scripts.
<i>dnsServerAddress</i>	The DNS server IP address.
<i>defaultDomain</i>	Optional. The default DNS domain
<i>loginInfo</i>	A frame specifying the login information. This frame contains the following three slots:
<i>interpreterSymbol</i>	A symbol identifying which interpreter to use. Use 'default for the default interpreter.
<i>loginScriptTimeout</i>	The number of seconds to wait for input.
<i>loginInstructions</i>	An array of frames interpreted by the default interpreter. The frames are described in the next section.

Login Script Frames

The default login script interpreter accepts an array of frames, each of which must have a `'type'` slot that specifies the type of information contained in the frame. Table 1-4 shows the frame types that you can use in your login scripts.

Table 1-4 Login script frames

Frame format	Description
<code>{type: 'waitFor', data: "data to wait for"}</code>	Waits for the string in the <code>data</code> slot. The string can contain any Unicode character. The string is converted to ASCII using the standard Macintosh encoding.
<code>{type: 'send', data: "data to send"}</code>	Sends the string specified in the <code>data</code> slot. The string can contain any Unicode character. The string is converted to ASCII using the standard Macintosh encoding. Note that a newline is not automatically sent.
<code>{type: 'sendCR'}</code>	Sends a newline (0x10).
<code>{type: 'pause', data: 1}</code>	Pauses the input script for the number of seconds specified in the <code>data</code> slot.
<code>{type: 'userName'}</code>	Sends the string in the <code>userName</code> slot of the link entry information frame. Note that a newline is not automatically sent.
<code>{type: 'password'}</code>	Sends the string in the <code>password</code> slot of the link entry information frame. Note that a newline is not automatically sent.

Table 1-4 Login script frames (continued)

Frame format	Description
{type: 'localAddress}	Waits for and reads in an IP address. The value of the link entry frame's <code>localAddress</code> slot is overridden with this address value. Use with SLIP connections to obtain the dynamically assigned address.
{type: 'gatewayAddress}	Waits for and reads in an IP address. The value of the link entry frame's <code>gatewayAddress</code> slot is overridden with this address value. Use with SLIP connections to obtain the dynamically assigned address.
{type: 'sendPrompt data: "prompt to user"}	Prompts the user to enter a string, such as a SecurID number, which is then sent. The prompt slip displays the string that you pass in as data.

The Domain Name Service Callback Function Format

The callback function for the `DNSCancelRequests` function does not receive any parameter values.

The callback function that you supply to each of the other domain name service calls must be declared with the following format:

```
func callbackFcn(resultsArray, resultCode);
```

The system software calls `callbackFcn` after the Newton Internet Enabler function has completed.

<i>resultsArray</i>	An array of zero or more DNS results frames. The format of a DNS results frame is described in the next section, "The DNS Results Frame."
<i>resultCode</i>	One of the result codes described in the section "DNS Error Codes" on page 1-32.

The DNS Results Frame

The DNS results frame contains a number of slots that describe the DNS operation. Different slot values are filled in by each of the DNS operations. The possible slot types are shown here:

```
{
    type resultType,
    targetDomainName domainNameString,
    resultDomainName domainNameString,
    targetIPAddress addressArray,
    resultIPAddress addressArray
}
```

Each results frame contains a `type` slot and at least one result slot. Most results frames contain the `targetDomainName` slot; however, this is not guaranteed. Table 1-5 shows which slot is guaranteed to be valid for each DNS operation.

Table 1-5 Result slots for each DNS operation

DNS operation	Results frame slot
DNSGetAddressFromName	resultIPAddress
DNSGetNameFromAddress	resultDomainName
DNSGetMailServerNameFromDomainName	resultDomainName
DNSGetMailAddressFromName	resultIPAddress

As you can see in Table 1-5, if the operation results in a domain name, the results frame contains one or more `resultDomainName` slots. If the operation results in an IP address, the results frame will instead contain one or more `resultIPAddress` slots.

For example, the `DNSGetAddressFromName` function returns a results array that looks something like this:

Newton Internet Enabler

```
[ {
  type kDNSAddressType,
  targetDomainName "newton.apple.com.",
  resultIPAddress [155,227,54,3]
} ]
```

In contrast, the `DNSGetNameFromAddress` function returns a results array that looks something like this

```
[{
  type kDNSDomainNameType,
  targetDomainName "newton.apple.com.",
  resultIPAddress [155,227,54,3]
} ]
```

Some DNS operations return a results array that contains more than one results frame. For example, a mail exchange operation can generate multiple mail exchange results frames.

The value types for each frame type are described below.

Slot descriptions

<i>resultType</i>	The type of result contained in the frame. This is either <code>kDNSAddressType</code> or <code>kDNSDomainNameType</code>. If the value is <code>kDNSAddressType</code>, the operation that generated this result frame resulted in an IP address; for example, the <code>DNSGetAddressFromName</code> function. If the value is <code>kDNSDomainNameType</code>, the operation that generated this result frame resulted in a domain name string; for example, the <code>DNSGetNameFromAddress</code> function.
<i>domainNameString</i>	The domain name used or resulting from the operation. For example "newton.apple.com."
<i>addressArray</i>	The IP address, specified as an array of four bytes. For example, [155, 227, 54, 3].

Link Controller Functions and Methods

This section describes the link controller functions and methods.

InetAddNewLinkEntry

linkID InetAddNewLinkEntry(*newLinkInfo*)

Installs a new link information entry on the user's Newton and returns the ID assigned to the entry.

newLinkInfo A link entry information frame, as described in "The Link Entry Information Frame" on page 1-51.

You can use the `InetAddNewLinkEntry` function to install a new link information entry in the Internet setup soup. The new link entry is created using the information supplied in the *newLinkInfo* frame. A new linkID is allocated for the entry. That link ID is returned as the function value.

The `InetAddNewLinkEntry` function adds the new entry to the soup on the default store. In contrast, the `InetAddNewLinkEntryInternal` function always adds the new entry to the soup on the internal store.

InetAddNewLinkEntryInternal

linkID InetAddNewLinkEntryInternal(*newLinkInfo*)

Installs a new link information entry on the user's Newton and returns the ID assigned to the entry.

Note

This function is the same as the `InetAddNewLinkEntry` function, except that `InetAddNewLinkEntryInternal` always creates the entry on the internal store. ♦

newLinkInfo A link entry information frame, as described in "The Link Entry Information Frame" on page 1-51.

You can use the `InetAddNewLinkEntryInternal` function to install a new link information entry in the Internet setup soup. The new link entry is created in the internal store using the information supplied in the

Newton Internet Enabler

newLinkInfo frame. A new linkID is allocated for the entry. That link ID is returned as the function value.

The `InetAddNewLinkEntryInternal` function adds the new entry to the soup on the internal store. In contrast, the `InetAddNewLinkEntryInternal` function adds the new entry to the soup on the default store.

InetCancelCurrentRequest

`InetCancelCurrentRequest(linkId)`

Cancels any active request on the specified link.

linkId The ID of the link that you want to cancel. Specify `nil` to use the default link ID.

The `InetCancelCurrentRequest` function cancels any active requests on the link specified by *linkId*.

InetCancelLink

`InetCancelLink(linkId, clientContext, clientCallback)`

Cancels an `InetGrabLink` operation that is in progress.

linkID The ID of the link that you want to cancel. Specify `nil` to use the default link ID.

clientContext The client context, as described in the section “The Client Context Parameter” on page 1-49.

clientCallback The client callback function, as described in the section “The Link Controller Callback Function Format” on page 1-50.

The `InetCancelLink` function cancels an `InetGrabLink` request that is in progress.

The `InetGrabLink` function is described on page 1-64.

InetDisconnectLink

`InetDisconnectLink(linkId, clientContext, clientCallback)`

Disconnects a link no matter how many clients are using the link.

<i>linkID</i>	The ID of the link that you want to cancel. Specify <code>nil</code> to use the default link ID.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Link Controller Callback Function Format” on page 1-50.

The `InetDisconnectLink` function disconnects a link directly without needing to close all of the applications that are using it. Note that this function ignores the hang-up time preference set by the user in the Internet Setup application.

WARNING

The `InetDisconnectLink` function is intended for use only by special purpose utility programs. Do not use this function unless you are certain that it the right thing to do.



InetDisplayStatus

statusView `InetDisplayStatus(linkId, statusView, status)`

Displays status information about a link on the user’s Newton screen.

<i>linkID</i>	The ID of the link for which you want to display status information. This must be an active link. Specify <code>nil</code> to use the default link ID.
<i>statusView</i>	The view to use for displaying the status. The view must be based on <code>protoStatusTemplate</code> .

If you provide a template, `InetDisplayStatus` opens the view for your. If you supply `nil` as the value of this

Newton Internet Enabler

parameter, `InetDisplayStatus` creates and opens a new view for you.

status

A status frame, such as the one passed to your `InetGrabLink` callback function. The format of this frame is described in the section “The Link Controller Status Frame” beginning on page 1-50.

If you supply `nil` as the value of this parameter and *statusView* is not `nil`, `InetDisplayStatus` removes the status display from the screen.

The `InetDisplayStatus` function displays link status information on the Newton screen. The view also contains a Stop button that the user can tap to cancel a grab link operation. If the grab gets cancelled, you can keep the view open and continue to display your own application status. To do so, follow these steps:

1. Set the value of the `appSymbol` slot in the status view to reference your application. For example:

```
myStatusView.appSymbol := kAppSymbol;
```

2. Implement a cancel script in your base view. When the user taps the Stop button, the status view calls a `'CancelScript(reason)` script in your base view. You can implement this script to respond to the stop button.

The `InetDisplayStatus` function returns the view that it used to display the status. You can use this for future calls to this function.

Note

To create a new status view for display on the screen, specify `nil` as the value of the *statusView* parameter. Newton Internet Enabler will create the view and return it as the function value.

To remove the status view from the screen, specify `nil` as the value of the *status* parameter and specify a view (not `nil`) as the value of the *statusView* parameter. ♦

Newton Internet Enabler

InetGetAllLinksStatus

linksStatusFrame InetGetAllLinksStatus()

The `InetGetAllLinksStatus` function returns a frame that specifies the status of all known links. If there are no available links, the `InetGetAllLinksStatus` function returns `nil` or a frame containing empty arrays.

WARNING

Since the user can add or remove available links at any time by using the Internet Setup application, you must be careful about caching the results of the `InetGetAllLinksStatus` function. ▲

The *linksStatusFrame* contains three slots, each of which is an array with one entry for each available link.

Slot descriptions

<i>LinkIDs</i>	An array that contains the ID of each available link.
<i>statuses</i>	An array that contains the status code for each available link. Each entry in this array is a status code value, as described in the section “Status Code Constants” on page 1-31.
<i>names</i>	An array that contains the name for each available link. Each entry in this array is the string name that the user specified when defining the link configuration in the Internet Setup application.

The following is an example of the *linksStatusFrame*:

```
{
  LinkIDs [1, 2, 3]
  statuses ['connected', 'idle', 'idle'],
  names ["Best", "Scruznet", "Compuserve"]
}
```

Newton Internet Enabler

Note

The `InetGetAllLinksStatus` function is a synchronous call that raises an exception if it encounters any problems. ♦

InetGetDefaultLinkID

linkID `InetGetDefaultLinkID()`

The `InetGetDefaultLinkID` function returns the ID of the default link. This is the ID that the Newton system software uses when you specify `nil` as the value of the *linkID* parameter for one of the other link controller functions.

InetGetLinkEntry

linkEntry `InetGetLinkEntry(linkID)`

Returns the link entry information frame associated with *linkID*.

linkID The ID of the link. Specify `nil` to use the default link ID.

The `InetGetLinkEntry` function returns the link entry information frame for the link *linkID*. For information about the format of link entry information frames, see “The Link Entry Information Frame” on page 1-51.

InetGetLinkStatus

`InetGetLinkStatus(linkID)`

Returns the status of a link.

linkID The ID of the link. Specify `nil` to use the default link ID.

The `InetGetLinkStatus` function returns the status of the link *linkID*. The status value is one of the status code constants, as described in the section “Status Code Constants” on page 1-31..

Note

The `InetGetLinkStatus` function is a synchronous function that raises an exception if it encounters any problems. ♦

Newton Internet Enabler

InetGrabLink

`InetGrabLink(linkId, clientContext, clientCallback)`

Provides access to a link.

<i>linkID</i>	The ID of the link that you want to access. Specify <code>nil</code> to use the default link ID.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Link Controller Callback Function Format” on page 1-50.

The `InetGrabLink` function attempts to access a link and calls your callback function with the status and error code for that operation.

The Newton system software calls your *clientCallback* function repeatedly while attempting to connect, supplying you with the current status. The `InetGrabLink` operation does not complete until the returned status is 'connected.

If the `InetGrabLink` operation fails, the `err` parameter to the *clientCallback* function indicates the reason. Otherwise, the value of the `err` parameter is `nil`.

InetOpenConnectionSlip

view `InetOpenConnectionSlip(linkId, clientContext, clientCallback)`

Opens a connection slip for the link. The user can change link information with the slip.

<i>linkID</i>	The ID of the link for which you want to open a connection slip. Specify <code>nil</code> to use the default link ID.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.

Newton Internet Enabler

clientSlipCallback A client callback function that receives a single parameter. This parameter is a symbol with one of the following values: 'connect or 'close.

The `InetOpenConnectionSlip` function displays a connection slip on the screen to allow the user to change to a different link or override the current worksite or phone number information. The user can complete the slip by tapping the Connect button or by tapping the Close box.

Your *clientCallback* function is called after the slip has been completed by the user. If your callback is called with the 'close symbol, it means that the user has canceled the connection by tapping the Close box in the slip. If your callback is called with the 'connect symbol, it means that the user has tapped the Connect button in the slip and that you should call `InetGrabLink` to proceed with the connection process.

If the link is already open, `InetOpenConnectionSlip` does not open a slip. In this case, `InetOpenConnectionSlip` calls your callback function with 'connect, which means that you can proceed as if the user had tapped the Connect button.

The `InetOpenConnectionSlip` function returns the open slip view or nil if it does not open a slip.

IMPORTANT

Apple recommends that you call `InetOpenConnectionSlip` before calling `InetGrabLink` to allow users the opportunity to change the connection settings. ▲

InetReleaseLink

`InetReleaseLink(linkId, clientContext, clientCallback)`

Relinquishes access to a link.

linkID The ID of the link that you want to release. Specify nil to use the default link ID.

Newton Internet Enabler

<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Link Controller Callback Function Format” on page 1-50.

The `InetReleaseLink` function releases your access to a link. If the link does not have any additional clients, it may close down.

Note

If the user has enabled a release link timeout in the link entry, then `InetReleaseLink` does not necessarily drop the link, even if it does not have any additional clients. When the count goes to zero and a release link timeout has been enabled, the Newton Internet Enabler software displays an indicator on the user’s screen (a blinking star at the top center of the screen). The user can tap that indicator to open a slip that can be used to release the link. If another `InetGrabLink` call is issued for the link before the timeout completes, the link is not released. ♦

Your *clientCallback* function is called after the link has been released. The status of the link at that time will depend on its use by other applications.

IMPORTANT

Apple recommends that you call `InetReleaseLink` whenever you no longer need to perform communications over the link for fifteen minutes or longer. ▲

InetSetDefaultLinkID

`InetSetDefaultLinkID(linkID)`

Establishes which link is the default link.

<i>linkID</i>	The ID of the link that you want to become the default link ID.
---------------	---

The `InetSetDefaultLinkID` function establishes *linkID* as the default link ID. The Newton Internet Enabler software uses the default link ID whenever

Newton Internet Enabler

you pass `nil` as the value of a `linkID` parameter. For more information about the default link ID, see “The Link Identifier Parameter” on page 1-48.

Domain Name Service Functions and Methods

This section describes the functions and methods that you can use to access the domain name service.

DNSCancelRequests

`DNSCancelRequests(clientContext, clientCallback)`

Cancels outstanding domain name server requests.

clientContext The client context, as described in the section “The Client Context Parameter” on page 1-49.

clientCallback The client callback function. This function is called with no parameters, in contrast to the other DNS callback functions.

The `DNSCancelRequests` function cancels any outstanding domain name service requests that have been made by the client with context *clientContext*. You must supply the same value for *clientContext* as you did when you made the DNS request.

Newton Internet Enabler will not call the callback functions for any DNS requests that get cancelled.

WARNING

The callback function that you supply to `DNSCancelRequests` does not have any parameters. This is different from the other DNS callback functions. ♦

DNSGetAddressFromName

`DNSGetAddressFromName(nameString, clientContext, clientCallback)`

Translates a domain name into its corresponding Internet address.

Newton Internet Enabler

<i>nameString</i>	An Internet domain name string.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Domain Name Service Callback Function Format” on page 1-55.

The `DNSGetAddressFromName` function resolves the domain name *nameString* into an IP address. `DNSGetAddressFromName` fills in the `resultIPAddress` slot in a DNS results frame, as described in the section “The Domain Name Service Callback Function Format” beginning on page 1-55, and calls your callback function with the result code and that frame as parameters. The following is an example of a results frame for the `DNSGetAddressFromName` function:

```
{
type kDNSAddressType,
targetDomainName "newton.apple.com.",
resultIPAddress [155,227,54,3],
}
```

The result code is `nil` if the function succeeded; otherwise, the result code is one of the error codes described in the section “DNS Error Codes” beginning on page 1-32.

If *nameString* ends with the period (`' . '`) character, the string is assumed to be in conformance with RFC 1123. If *nameString* does not end with the period character, `DNSGetAddressFromName` attempts to complete the name as follows:

- `DNSGetAddressFromName` first appends a period to a copy of *nameString* and attempts name resolution with that string.
- If that request is not successful, `DNSGetAddressFromName` appends the local domain name and a period to a copy of *nameString* and attempts name resolution with that string.

DNSGetMailAddressFromName

`DNSGetMailAddressFromName(nameString, clientContext, clientCallback)`

Translates a domain name into the Internet address for a mail server that serves that domain.

<i>nameString</i>	An Internet domain name string.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Domain Name Service Callback Function Format” on page 1-55.

The `DNSGetMailAddressFromName` function returns the IP address of a mail server that serves the domain specified by *nameString*.

`DNSGetMailAddressFromName` fills in the `resultIPAddress` slot in a DNS results frame, as described in the section “The Domain Name Service Callback Function Format” beginning on page 1-55, and calls your callback function with the result code and that frame as parameters. The following is an example of a results frame for the `DNSGetMailAddressFromName` function:

```
{
type kDNSAddressType,
targetDomainName "mail.newton.apple.com.",
resultIPAddress [155,227,54,3],
}
```

The result code is `nil` if the function succeeded; otherwise, the result code is one of the error codes described in the section “DNS Error Codes” beginning on page 1-32.

If *nameString* ends with the period (‘.’) character, the string is assumed to be in conformance with RFC 1123. If *nameString* does not end with the period character, `DNSGetAddressFromName` attempts to complete the name as follows:

Newton Internet Enabler

- `DNSGetMailAddressFromName` first appends a period to a copy of *nameString* and attempts name resolution with that string.
- If that request is not successful, `DNSGetMailAddressFromName` appends the local domain name and a period to a copy of *nameString* and attempts mail server resolution with that string.

If there is more than one mail server for the domain specified by *nameString*, the results array contains multiple `resultDomainName` frames, one for each mail server. The frames are ordered in the results frame according to the preference order of the mail exchange resource record.

DNSGetMailServerNameFromDomainName

```
DNSGetMailServerNameFromDomainName( nameString,
                                       clientContext, clientCallback )
```

Translates a domain name into the domain name for a mail server that serves that domain.

<i>nameString</i>	An Internet domain name string.
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.
<i>clientCallback</i>	The client callback function, as described in the section “The Client Callback Parameter” on page 1-49.

The `DNSGetMailServerNameFromDomainName` function returns the domain name for a mail server that serves the domain specified by *nameString*. `DNSGetMailServerNameFromDomainName` fills in the `resultDomainName` slot in a DNS results frame, as described in the section “The Domain Name Service Callback Function Format” beginning on page 1-55, and calls your callback function with the result code and that frame as parameters. The following is an example of a results frame for the `DNSGetMailServerNameFromDomainName` function:

```
{
type kDNSDomainNameType,
targetDomainName "newton.apple.com.",
```

Newton Internet Enabler

```
resultDomainName "mail.newton.apple.com.",
}
```

The result code is `nil` if the function succeeded; otherwise, the result code is one of the error codes described in the section “DNS Error Codes” beginning on page 1-32.

If *nameString* ends with the period (‘.’) character, the string is assumed to be in conformance with RFC 1123. If *nameString* does not end with the period character, `DNSGetMailServerNameFromDomainName` attempts to complete the name as follows:

- `DNSGetMailServerNameFromDomainName` first appends a period to a copy of *nameString* and attempts name resolution with that string.
- If that request is not successful, `DNSGetMailServerNameFromDomainName` appends the local domain name and a period to a copy of *nameString* and attempts name resolution with that string.

If there is more than one mail server for the domain specified by *nameString*, the results array contains multiple `resultDomainName` frames, one for each mail server. The frames are ordered in the results frame according to the preference order of the mail exchange resource record. This is the order in which a mail application should attempt to connect to the SMTP ports of these servers.

DNSGetNameFromAddress

`DNSGetNameFromAddress(address, clientContext, clientCallback)`

Translates an Internet address into its corresponding domain name.

<i>address</i>	An Internet IP address, specified as a NewtonScript array of four integer values. For example: [155, 227, 54, 3].
<i>clientContext</i>	The client context, as described in the section “The Client Context Parameter” on page 1-49.

Newton Internet Enabler

clientCallback The client callback function, as described in the section “The Client Callback Parameter” on page 1-49.

The `DNSGetAddressFromName` function finds the domain name string for the IP *address*. `DNSGetNameFromAddress` fills in the `resultDomainName` slot in a DNS results frame, as described in the section “The Domain Name Service Callback Function Format” beginning on page 1-55, and calls your callback function with the result code and that frame as parameters. The following is an example of a results frame for the `DNSGetAddressFromName` function:

```
{
type kDNSDomainNameType,
targetIPAddress [155, 227, 54, 3],
resultDomainName "newton.apple.com.",
}
```

The result code is `nil` if the function succeeded; otherwise, the result code is one of the error codes described in the section “DNS Error Codes” beginning on page 1-32.

If there is more than one domain name for the address specified by *address*, the results array contains multiple results frames, each with a `resultDomainName` slots.

Newton Internet Enabler Error Functions

This section describes the functions you can use to work with errors generated by the Newton Internet Enabler.

InetGetErrorString

String `InetGetErrorString(errorNum)`

Retrieves the error message string for the specified *errorNum*.

errorNum An exception frame.

Newton Internet Enabler

The `InetGetErrorString` function returns a message string that corresponds to the error specified by `errorNum`.

If *errorNum* is not a Newton Internet Enabler error, this function calls the ROM error function to retrieve the message string. If *errorNum* is not a Newton Internet Enabler error and is not in the ROM error table, `InetGetErrorString` returns a string containing *errorNum*.

InetGetExceptionError

Integer `InetGetExceptionError(exception)`

Returns the error number corresponding to the exception.

exception An exception frame.

The `InetGetExceptionError` function returns the error number corresponding to the *exception*. You can call the `InetGetErrorString` function with the returned value to convert the error code into a message string.

Newton Internet Enabler Options

This section describes the options that you can use to control Newton Internet Enabler.

Inet Tool Expedited Data Transfer ('iexp') Option

The Inet Tool expedited data transfer option is used for the expedited transmission of an unsigned data byte. You can use this option with an `Output` request to your endpoint to cause the data in that request to be sent immediately. You typically use this to send a break character or a similar indicator.

Here is an example of this option:

```
{
    label:      "iexp",
    type:      'option,
```

Newton Internet Enabler

```
opCode:      opSetRequired,
result:      nil,
form:        'template,
data:
{
    arglist:
    [
        15          // expeditedData byte
    ],
    typelist:
    [
        'struct,
        'byte
    ],
},
}
```

The data slots in the expedited data transfer option frame are described in Table 1-6.

Table 1-6 Inet tool expedited data transfer option data slots

Option field	Description
expeditedData	The data byte that was received or is to be sent.

Inet Tool Physical Link Identifier ('ilid') Option

The Inet physical link identifier option is used to set or retrieve the physical link identifier.

Here is an example of using this option to set the physical link identifier:

```
{
    label:      "ilid",
    type:       'option,
```

Newton Internet Enabler

```
opCode:      opSetRequired,
result:      nil,
form:        'template,
data:
{
    arglist:
    [
        linkID
    ],
    typelist:
    [
        'struct,
        'ulong
    ],
},
}
```

The data slots in the physical link identifier option frame are described in Table 1-7.

Table 1-7 Inet tool physical link identifier option data slots

Option field	Description
linkID	The link identifier.

Inet Tool Local Port ('ilpt') Option

The Inet Tool local port option is used to set or retrieve the Internet port number for a transport service. The rules shown in Table 1-9 apply to port number assignments.

Here is an example of this option to set the port number:

```
{
    label:      "ilpt",
```

Newton Internet Enabler

```
type:      'option,
opCode:    opSetRequired,
result:    nil,
form:      'template,
data:
{
  arglist:
  [
    7,          // inetPortNumber
    nil,        // useDefault
  ],
  typelist:
  [
    'struct,
    'short,
    'boolean
  ],
},
}
```

The data slots in the local port option frame are described in Table 1-8.

Table 1-8 Inet tool local port option data slots

Option field	Description
inetPortNumber	The reserved port number for this service. This value is used as described in Table 1-9
useDefault	A Boolean value that applies only to <i>connect</i> binds for the UDP transport service type. If useDefault is true, the default UDP port number is used.

Newton Internet Enabler

Table 1-9 shows the use of the `inetPortNumber` slot, based on the service type and operation.

Table 1-9 Use of the port number by the Inet tool

Service type and operation	Description
Connect over TCP link	TCP picks this port; no need to set.
Listen over TCP link	The port on which to listen. Specify 0 to indicate listening on all ports or use one of the port numbers as specified in <i>IEFT Assigned Numbers RFC</i> .
Connect over UDP link	The port to bind to locally. Specify <code>useDefault:true</code> to indicate that Newton Internet Enabler should choose the port number for you, in which case the assigned value will be returned in the option.
Listen over UDP link	The port on which to listen. Specify 0 to indicate listening on all ports or use one of the port numbers as specified in <i>IEFT Assigned Numbers RFC</i> .

Inet Profile ('iprf') Option

The Inet profile option is used to retrieve the the local and gateway IP addresses used by your endpoint.

Here is an example of this option to get the IP addresses:

```
{
  label:      "iprf",
  type:      'option',
  opCode:    opGetCurrent,
  data:
  {
    arglist:
    [
```

Newton Internet Enabler

```

        [0,0,0,0],      // local host (Newton) IP address
        [0,0,0,0],      // gateway host IP address
    ],
    typelist:
    [
        'struct,
        ['array, 'byte, 4],
        ['array, 'byte, 4],
    ],
    },
}

```

Inet Tool TCP Remote Socket ('itrs') Option

The Inet Tool TCP remote socket option is used to set or retrieve the parameters of the remote host. If you are sending a **Connect** request over a TCP link, you must use this option to retrieve the remote socket address; if you are sending a **Listen** request over a TCP link, you can use this option to retrieve the address of the sender of the data.

Here is an example of using this option to set the TCP remote socket:

```

{
    label:      "itrs",
    type:      'option,
    opCode:    opSetRequired,
    result:    nil,
    form:      'template,
    data:
    {
        arglist:
        [
            [130,43,2,2],      // hostAddress
            7,                  // InetPortNumber
        ],
    },
}

```

Newton Internet Enabler

```
typelist:
[
    'struct,
    ['array, 'byte, 4],
    'short
]
},
}
```

The data slots in the TCP remote socket option frame are described in Table 1-10.

Table 1-10 Inet tool TCP remote socket option data slots

Option field	Description
hostAddress	Internet address of remote host IP address, expressed as four single-byte values.
InetPortNumber	Reserved Internet port identifier.

Inet Tool Transport Service Type ('itsv') Option

The Inet Tool transport service type option is used to specify the transport service type associated with a link.

Here is an example of using this option to set the transport service type:

```
{
    label:      "itsv",
    type:       'option,
    opCode:     opSetRequired,
    result:     nil,
    form:       'template,
    data:
    {
```

Newton Internet Enabler

```
    arglist:
    [
        kTCP          // transportService
    ],
    typelist:
    [
        'struct,
        'ulong
    ],
    },
}
```

The data slots in the link service type option frame are described in Table 1-11.

Table 1-11 Inet tool link service type option data slots

Option field	Description
transportService	The transport service type. Use one of the constants described in the section “Transport Service Type Constants” beginning on page 1-31.

Inet Tool UDP Destination Socket ('iuds') Option

The Inet Tool UDP destination socket option is used to set or retrieve the Internet destination host Internet socket address that is used for data transmission over a UDP link.

Here is an example of using this option to retrieve the current UDP destination address:

```
{
    label:      "iuds",
    type:      'option,
```

Newton Internet Enabler

```
opCode:      opSetRequired,
result:      nil,
form:        'template,
data:
{
  arglist:
  [
    [0,0,0,0]      // hostAddress
    0,              // InetPortNumber
  ],
  typelist:
  [
    'struct,
    ['array, 'byte, 4],
    'short
  ]
}
```

The data slots in the UDP destination socket option frame are described in Table 1-12.

Table 1-12 Inet tool UDP destination socket option data slots

Option field	Description
hostAddress	The destination IP address, expressed as four single-byte values.
InetPortNumber	The reserved Internet port identifier.

Inet Tool UDP Source Socket ('iuss') Option

The Inet Tool UDP source socket option is used to retrieve the host Internet socket address that sent a datagram received by your application.

Newton Internet Enabler

Here is an example of using this option to retrieve the UDP source socket:

```
{
  label:      "iuds",
  type:      'option,
  opCode:    opGetCurrent,
  result:    nil,
  form:      'template,
  data:
  {
    arglist:
    [
      [0,0,0,0],          // hostAddress
      port,                // InetPortNumber
    ],
    typelist:
    [
      'struct,
      ['array, 'byte, 4],
      'short
    ]
  },
}
```

The data slots in the UDP source socket option frame are described in Table 1-13.

Table 1-13 Inet tool UDP source socket option data slots

Option field	Description
hostAddress	The source IP address, expressed as four single-byte values.
InetPortNumber	The reserved Internet port identifier.

Newton Internet Enabler Exceptions

Any of the Newton Internet Enabler functions that receive a link identifier as a parameter can throw an exception if the link ID is not valid. The exception frame is as follows:

```
{
  type: |evt.ex.comm|,
  error: kInetErrNoSuchLinkID,
}
```

This exception is raised in response to two conditions:

- You passed in a link ID that does not exist.
- You passed in `nil` as the link ID and there are not any links defined.

Summary of Newton Internet Enabler

Link Controller Errors

constant kInetErrNoSuchLinkID	:= -60501;
constant kInetErrLinkDisconnected	:= -60504;
constant kInetErrConnectLinkFailed	:= -60505;

DNS Errors

constant kDNSErrNoAnswerFoundYet	:= -60751;
constant kDNSErrInternalErr	:= -60752;
constant kDNSErrNameSyntaxErr	:= -60791;
constant kDNSErrNoNameServer	:= -60794;
constant kDNSErrAuthNameErr	:= -60795;
constant kDNSErrNoAnswerErr	:= -60796;
constant kDNSErrNonexistentDomain	:= -60797;

Newton Internet Enabler

constant kDNSErrOutOfMemory	:= -60798;
constant kDNSErrCouldNotContactServer	:= -60800;
constant kDNSErrNoServersAvailable	:= -60801;
constant kDNSErrRequestFormatErr	:= -60802;
constant kDNSErrServerInternalErr	:= -60803;
constant kDNSErrServerNotImplemented	:= -60804;
constant kDNSErrServerRefused	:= -60805;
constant kDNSErrUnknownServerError	:= -60806;
constant kDNSErrNoResponseFromServer	:= -60814;
constant kDNSErrNoResponseFromAnyServer	:= -60815;

Newton Internet Enabler PPP Errors

constant kInetToolErrPppAuthSetupFailed	:= -60084;
constant kInetToolErrPppAuthReqFailed	:= -60085;

Newton Internet Enabler Lower-Level Tool Errors

constant kInetToolErrBindFailed	:= -60001;
constant kInetToolErrIPBindFailed	:= -60002;
constant kInetToolErrPushModule	:= -60004;
constant kInetToolErrIlink	:= -60005;
constant kInetToolErrNetActivateReq	:= -60006;
constant kInetToolErrTCPBind	:= -60007;
constant kInetToolErrGetRequest	:= -60008;
constant kInetToolErrPutRequest	:= -60009;
constant kInetToolErrConnect	:= -60010;
constant kInetToolErrDLAttach	:= -60011;
constant kInetToolErrBind	:= -60012;
constant kInetToolErrOpenLink	:= -60013;
constant kInetToolErrUnlink	:= -60014;
constant kInetToolErrOutOfPhase	:= -60015;
constant kInetToolErrAddRoute	:= -60016;

Newton Internet Enabler

```

constant kInetToolErrListen           := -60017;
constant kInetToolErrLinkNotOpened    := -60018;
constant kInetToolErrDriverNotOpened  := -60019;
constant kInetToolErrStreamNotOpened  := -60020;
constant kInetToolErrBindReqFailed    := -60021;
constant kInetToolErrConnResReqFailed := -60022;

constant kInetToolErrMemAlloc          := -60023;
constant kInetToolErrMsgType           := -60024;
constant kInetToolErrNoDevice          := -60025;
constant kInetToolErrIllegalOpenOnStream := -60026;
constant kInetToolErrReqInInvalidState := -60027;
constant kInetToolErrPrimitiveTooSmall := -60028;
constant kInetToolErrPrimitiveOutOfRange := -60029;
constant kInetToolErrPrimitiveOnInvalidStr := -60030;
constant kInetToolErrMessageTooLong    := -60031;
constant kInetToolErrNetworkAlreadyActive := -60032;
constant kInetToolErrNetworkNumberInvalid := -60033;
constant kInetToolErrUnsupportedIoctl   := -60034;
constant kInetToolErrStreamAlreadyAttached := -60035;
constant kInetToolErrUnknownMuxIndex    := -60036;
constant kInetToolErrNetworkIsInactive  := -60037;
constant kInetToolErrBogusConnection    := -60038;
constant kInetToolErrInvalidBillingMode := -60039;
constant kInetToolErrNoTrigSelectedInAlarm := -60040;
constant kInetToolErrInvalidTrigSize     := -60041;
constant kInetToolErrInvalidConnectionRef := -60042;
constant kInetToolErrIllegalMdataInPrim := -60043;
constant kInetToolErrMissingMdataInPrim := -60044;
constant kInetToolErrInvalidSegmentedPrim := -60045;
constant kInetToolErrInvalidNPIVersion  := -60046;
constant kInetToolErrInvalidAddress      := -60047;
constant kInetToolErrOutOfTCPPortNumbers := -60048;

```

Newton Internet Enabler

```

constant kInetToolErrSocketInUse           := -60049;
constant kInetToolErrReservedPortNumber    := -60050;
constant kInetToolErrExpDataNotSupported    := -60051;

constant kInetToolErrRedundentRequest       := -60052;
constant kInetToolErrUnexpectedDLPrim      := -60053;
constant kInetToolErrUnexpectedTPIPrim     := -60054;
constant kInetToolErrUnexpectedNPIPrim     := -60055;
constant kInetToolErrUnknownTPIErrorCode   := -60056;

constant kInetErrStreamInoperative         := -60057;
constant kInetErrUnexpectedMgmntEvent      := -60058;

constant kInetErrLinkActivationNotConfirmed := -60059;
constant kInetErrLinkDeactivationNotConfirmed := -60060;
constant kInetErrLinkTerminateAckNotReceived := -60061;
constant kInetErrNetworkDifferentFromTheReq := -60062;
constant kInetErrConnLossFromTerminateReq  := -60063;
constant kInetErrConnLossFromConfigureRequest := -60064;
constant kInetErrConnLossFromConfigureReject := -60065;
constant kInetErrConnLossFromConfigureAck   := -60066;
constant kInetErrConnLossFromConfigureNack  := -60067;
constant kInetErrConnLossFromCodeReject     := -60068;
constant kInetErrConnLossFromMaxSetUpAttempts := -60069;
constant kInetErrConnLossFromMandatoryFrameSize := -60070;
constant kInetErrConnLossFromMandatoryAuthRej := -60071;
constant kInetErrConnLossFromRemoteInitAuthFail := -60072;
constant kInetErrConnLossFromLocalInitAuthFail := -60073;
constant kInetErrConnLossFromLowerLinkFailure := -60074;
constant kInetErrConnLossUnspecifiedReason  := -60075;
constant kInetErrConnLossFromMaxRemoteInitAuthFail
                                           := -60076;
constant kInetErrConnLossFromMaxLocalInitAuthFail

```

Newton Internet Enabler

```

:= -60077;
constant kInetErrConnLossFromMngmntDeactivation:= -60078;
constant kInetErrConnLossFromNoIpAddr := -60079;

constant kInetToolErrDataLinkLevel := -60080;
constant kInetToolErrSubsBind := -60081;
constant kInetToolErrNetDeActivateReq := -60082;
constant kInetToolErrNetworkLevel := -60083;

```

Link Controller Functions and Methods

```

linkId InetAddNewLinkEntry(newLinkInfo);
linkId InetAddNewLinkEntryInternal(newLinkInfo);
InetCancelCurrentRequest(linkId);
InetCancelLink(linkId, clientContext, clientCallback);
InetDisconnectLink(linkId, clientContext, clientCallback);
statusView InetDisplayStatus(linkId, statusView, status);
linksStatusFrame InetGetAllLinksStatus();
linkID InetGetDefaultLinkID();
linkEntry InetGetLinkEntry(linkID);
InetGetLinkStatus(linkID);
InetGrabLink(linkId, clientContext, clientCallback);
view InetOpenConnectionSlip(linkId, clientContext, clientCallback);
InetReleaseLink(linkId, clientContext, clientCallback);
InetSetDefaultLinkID(linkId);

```

DNS Functions and Methods

```

DNSCancelRequests(clientContext, clientCallback)
DNSGetAddressFromName(nameString, clientContext, clientCallback);
DNSGetMailAddressFromName(nameString, clientContext, clientCallback);

```

Newton Internet Enabler

```
DNSGetMailServerNameFromDomainName(nameString,  
                                     clientContext, clientCallback);  
DNSGetNameFromAddress(address, clientContext, clientCallback);
```

Error-handling Functions and Methods

```
String InetGetErrorString(errorNum);  
Integer InetGetExceptionError(exception);
```

Exceptions

| evt.ex.comm |

Index

A

application-related error codes 1-38

B

binding
 local port numbers for 1-18
binding endpoints with links 1-17

C

callback functions 1-9
 example of 1-10, 1-28
 for DNS functions 1-55
 for link controller functions 1-50
client callback 1-49
client context 1-27, 1-49
configuring links for endpoints 1-16
constants
 application-related error codes 1-38
 DNS error codes 1-32
 Inet tool error codes 1-35
 kEventToolSpecific 1-47
 PPP error codes 1-35
 PPP internal error codes 1-43
 status code 1-31
 tool-specific error codes 1-42
 transport service type 1-31
 UDP error codes 1-41

D

default link identifier 1-13, 1-48
displaying status 1-15
DNSCancelRequests function 1-28, 1-67
DNS error codes 1-32
DNSGetAddressFromName function 1-67
DNSGetMailAddressFromName function 1-69
DNSGetMailServerNameFromDomainName
 function 1-70
DNSGetNameFromAddress function 1-71
DNS results frame 1-56

domain name service
 about 1-2, 1-6
 using 1-28

E

endpoints
 and configuring links 1-16
 binding with links 1-17
 connecting with links 1-18
 disconnecting 1-25
 restrictions on use of 1-9
 using several with one link 1-8, 1-9
 with links 1-8
error codes
 application-related 1-38
 DNS 1-32
 Inet tool 1-35
 PPP 1-35
 PPP internal 1-43
 tool-specific 1-42
 UDP 1-41
events 1-46
 kEventToolSpecific 1-47
expedited data
 receiving 1-25
 sending 1-21

G

grabbing a link 1-5, 1-12

I

InetAddNewLinkEntry function 1-58
InetAddNewLinkEntryInternal function 1-58
InetCancelCurrentRequest function 1-59
InetCancelLink function 1-59
Inet communications tool
 about 1-2
InetDisconnectLink function 1-60
InetDisplayStatus function 1-15, 1-60
InetGetAllLinksStatus function 1-62

InetGetDefaultLinkID function 1-63
 InetGetErrorString function 1-72
 InetGetExceptionError function 1-73
 InetGetLinkEntry function 1-63
 InetGetLinkStatus function 1-63
 InetGrabLink function 1-12, 1-64
 InetOpenConnectionSlip function 1-64
 Inet profile option 1-77
 InetReleaseLink function 1-26, 1-65
 InetSetDefaultLinkID function 1-66
 Inet tool
 about 1-4
 Inet tool error codes 1-35
 Inet tool expedited data transfer option 1-73
 Inet tool local port option 1-75
 Inet tool physical link identifier option 1-74
 Inet tool TCP remote socket option 1-78
 Inet tool transport service type option 1-79
 Inet tool UDP destination socket option 1-80
 Inet tool UDP source socket option 1-81
 Internet Setup program 1-2

K

kEventToolSpecific event 1-47

L

link controller
 about 1-2, 1-4
 using 1-11
 link controller status frame 1-50
 link identifier 1-48
 linkInfo frame 1-51
 link information frame 1-51
 links
 binding with 1-17
 configuring with endpoints 1-16
 connecting to 1-18
 default identifier for 1-13
 grabbing 1-5, 1-12
 releasing 1-26
 sending data over 1-19
 status of 1-15
 with endpoints 1-8
 with multiple endpoints 1-8, 1-9
 local port numbers 1-18

M

methods and functions
 DNSCancelRequests 1-28, 1-67
 DNSGetAddressFromName 1-67
 DNSGetMailAddressFromName 1-69
 DNSGetMailServerNameFromDomainName 1-70
 DNSGetNameFromAddress 1-71
 InetAddNewLinkEntry 1-58
 InetAddNewLinkEntryInternal 1-58
 InetCancelCurrentRequest 1-59
 InetCancelLink 1-59
 InetDisconnectLink 1-60
 InetDisplayStatus 1-15, 1-60
 InetGetAllLinksStatus 1-62
 InetGetDefaultLinkID 1-63
 InetGetErrorString 1-72
 InetGetExceptionError 1-73
 InetGetLinkEntry 1-63
 InetGetLinkStatus 1-63
 InetGrabLink 1-12, 1-64
 InetOpenConnectionSlip 1-64
 InetReleaseLink 1-26, 1-65
 InetSetDefaultLinkID 1-66

N

Newton Internet Enabler
 about 1-2
 and communications endpoints 1-8
 callback functions 1-9
 components 1-3
 components of 1-2
 events 1-46
 options for 1-29
 using 1-7

O

options
 for each endpoint call 1-29
 for Newton Internet Enabler 1-73
 Inet profile ('iprf') 1-77
 Inet tool expedited data transfer ('iexp') 1-73
 Inet tool local port ('ilpt') 1-75
 Inet tool physical link identifier ('ilid') 1-74
 Inet tool TCP remote socket ('itrs') 1-78
 Inet tool transport service type ('itsv') 1-79
 Inet tool UDP destination socket ('iuds') 1-80
 Inet tool UDP source socket ('iuss') 1-81
 using 1-29

P

power management 1-27
PPP error codes 1-35
PPP internal error codes 1-43

R

receiving data 1-21
 expedited 1-25
 with TCP 1-24
 with UDP 1-22
releasing links 1-26
results frame 1-56

S

sending data 1-19
 expedited data 1-21
 with TCP 1-20
 with UDP 1-19
status code constants 1-31
status frame 1-50
status information 1-15

T

TCP
 receiving data with 1-24
 sending data with 1-20
tool-specific error codes 1-42
transport service type constants 1-31

U

UDP
 receiving data with 1-22
 sending data with 1-19
UDP error codes 1-41

I N D E X

This Apple manual was written, edited, and composed on a desktop publishing system using Apple Macintosh computers and FrameMaker software. Proof pages were created on an Apple LaserWriter Pro 630 printer. Final page negatives were output directly from the text and graphics files. Line art was created using Adobe™ Illustrator. PostScript™, the page-description language for the LaserWriter, was developed by Adobe Systems Incorporated.

Special thanks to Dan Chernikoff and Bruce Thompson.

Text type is Palatino® and display type is Helvetica®. Bullets are ITC Zapf Dingbats®.

LEAD WRITER
Gary Hillerson

PROJECT LEADER
Christopher Bey

ILLUSTRATOR
Peggy Kunz

EDITORS
Linda Ackerman, Anne Szabla

PROJECT MANAGER
Gerry Kane